

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

Oleh:

Muhammad Naufal Khalish NIM. 2410817120018

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Naufal Khalish
NIM : 2410817120018

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code	7
B. Output Program.....	38
C. Pembahasan.....	39
SOAL 2	61
A. Pembahasan.....	61
TAUTAN GIT	62

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Test Debugging Soal 1 XML	38
Gambar 2 Screenshot Hasil TestDebugging Soal 1 Compose.....	38

DAFTAR TABEL

Tabel 1. Source Code Song.kt XML.....	7
Tabel 2 Source Code SongData.kt XML	7
Tabel 3 Source Code DetailFragment.kt XML.....	8
Tabel 4 Source Code HighlightAdapter.kt XML.....	10
Tabel 5 Source Code HomeFragment.kt XML.....	10
Tabel 6 Source Code SongListAdapter.kt XML.....	13
Tabel 7 Source Code SettingsFragment.kt XML.....	14
Tabel 8 Source Code LanguageSettings.kt XML	15
Tabel 9 Source Code SongViewModel.kt XML.....	16
Tabel 10 SongViewModelFactory.kt XML	17
Tabel 11 Source Code MainActivity.kt XML	17
Tabel 12 Source Code MyApplication.kt XML	18
Tabel 13. Source Code Song.kt.....	27
Tabel 14. Source Code SongData.kt	27
Tabel 15. Source Code AppNavigation.kt	28
Tabel 16. Source Code DetailScreen.kt	29
Tabel 17. Source Code DetailScreen.kt	30
Tabel 18. Source Code SettingScreen.kt.....	33
Tabel 19. Source Code Theme.kt.....	34
Tabel 20. Source Code SongViewModel.kt	35
Tabel 21. Source Code SongViewModelFactory.kt	35
Tabel 22. Source Code MainActivity.kt	36

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - d. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out.

A. Source Code

XML

Song.kt

Tabel 1. Source Code Song.kt XML

1	package com.example.modul3xml.data
2	
3	data class Song(
4	val id: Int,
5	val title: String,
6	val year: String,
7	val meanId: String,
8	val meanEn: String,
9	val imageRes: Int,
10	val url: String
11	)

SongData.kt

Tabel 2 Source Code SongData.kt XML

1	package com.example.modul3xml.data
2	
3	import com.example.modul3xml.R
4	
5	object SongData {
6	
7	val SongList = listOf(
8	Song(
9	id = 1,
10	title = "Scott Street",
11	year = "2017",
12	meanId = "Lagu emosional tentang nostalgia dan perasaan terasing
13	dari Phoebe Bridgers.",
14	meanEn = "An emotional song about nostalgia and feeling alienated by
15	Phoebe Bridgers.",
16	imageRes = R.drawable.scootstreet,
17	url =
18	"https://open.spotify.com/track/21uFPefbgeR3QLVJWATlrr?si=ab2c7ad661ad4168"
19	),
20	Song(
21	id = 2,
22	title = "The Night We Met",
23	year = "2015",
24	meanId = "Balada melankolis dari Lord Huron tentang kerinduan untuk
25	kembali ke masa lalu.",
26	meanEn = "A melancholy ballad by Lord Huron about longing to return
27	to the past.",
28	imageRes = R.drawable.thenightwemet,
29	url =
30	"https://open.spotify.com/track/3hRV0jL3vUpRrcy398teAU?si=bfd097b1bc824f91"
31	),
32	Song(
33	id = 3,

28	title = "Hello",
29	year = "2015",
30	meanId = "Karya musik folk minimalis dari Ichiko Aoba dengan petikan gitar yang lembut.",
31	meanEn = "A folk rock song by Ichiko Aoba with a soft guitar.",
32	imageRes = R.drawable.hello,
33	url =
	"https://open.spotify.com/track/795bHfsDk7EC34IsLqaXfp?si=0a8eba42ee67448d"
34	),
35	Song(
36	id = 4,
37	title = "Light Song",
38	year = "2016",
39	meanId = "Karya musik yang menenangkan dengan harmoni vokal yang indah.",
40	meanEn = "A soothing musical piece featuring beautiful vocal harmonies.",
41	imageRes = R.drawable.lightson,
42	url =
43	"https://open.spotify.com/track/4KXZ4MJKT0RYGhijfQ08Jw?si=9e08213f078c4a8f"
44	),
45	Song(
46	id = 5,
47	title = "Childhood",
48	year = "1995",
49	meanId = "Lagu dari daniel.mp3 yang sangat personal tentang masa lalu saat kecil.",
50	meanEn = "A personal song about the past when I was a child.",
51	imageRes = R.drawable.childhood,
52	url =
53	"https://open.spotify.com/track/06dDLXf4gPbE02gvH8E6G9?si=1d289d479e104445"
54	)
55	fun findSongById(gameId: Int): Song? {
56	return SongList.find { it.id == gameId }
57	}
58	}

DetailFragment.kt

Tabel 3 Source Code DetailFragment.kt XML

1	package com.example.modul3xml.ui.detail
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.ViewGroup
6	import androidx.fragment.app.Fragment
7	import androidx.fragment.app.viewModels
8	import androidx.navigation.fragment.findNavController
9	import com.example.modul3xml.R
10	import com.example.modul3xml.data.SongData
11	import com.example.modul3xml.databinding.FragmentDetailBinding
12	import com.example.modul3xml.util.LanguageSettings
13	import com.example.modul3xml.viewmodel.SongViewModel
14	import com.example.modul3xml.viewmodel.SongViewModelFactory
15	import timber.log.Timber


```

16
17 class DetailFragment : Fragment() {
18
19     private var _binding: FragmentDetailBinding? = null
20     private val binding get() = _binding!!
21
22     private val viewModel: SongViewModel by viewModels {
23         SongViewModelFactory("Yuukosoo")
24     }
25
26     override fun onCreateView(
27         inflater: LayoutInflater,
28         container: ViewGroup?,
29         savedInstanceState: Bundle?
30     ) = FragmentDetailBinding.inflate(inflater, container, false).also {
31         _binding = it
32     }.root
33
34     override fun onViewCreated(view: android.view.View, savedInstanceState:
Bundle?) {
35         super.onViewCreated(view, savedInstanceState)
36
37         setupToolbar()
38         bindSongData()
39     }
40
41     private fun setupToolbar() {
42         binding.toolbarDetail.setNavigationOnClickListener {
43             findNavController().popBackStack()
44         }
45     }
46
47     private fun bindSongData() {
48         val songId = arguments?.getInt("songId") ?: -1
49         val song = viewModel.songs.value.find { it.id == songId }
50
51         if (song == null) {
52             binding.toolbarDetail.title = getString(R.string.title_detail)
53             Timber.w("DetailFragment: Lagu dengan ID $songId tidak ditemukan!")
54             return
55         }
56
57         Timber.d("DetailFragment: Menampilkan detail lagu ${song.title}")
58
59         val isIndonesian = LanguageSettings.isIndonesian(requireContext())
60
61         binding.toolbarDetail.title = song.title
62         binding.imageDetail.setImageResource(song.imageRes)
63         binding.textDetailTitle.text = song.title
64         binding.textDetailYear.text = song.year
65         binding.textDetailMean.text = if (isIndonesian) song.meanId else
song.meanEn
66     }
67
68     override fun onDestroyView() {
69         super.onDestroyView()
70         _binding = null
71     }
72 }

```

HighlightAdapter.kt

Tabel 4 Source Code HighlightAdapter.kt XML

1	package com.example.modul3xml.ui.home
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.example.modul3xml.data.Song
7	import com.example.modul3xml.databinding.ItemHighlightSongBinding
8	
9	class HighlightAdapter(10 private val song: List<Song> 11) : RecyclerView.Adapter<HighlightAdapter.HighlightViewHolder>() { 12 13 override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): HighlightViewHolder { 14 val binding = ItemHighlightSongBinding.inflate(15 LayoutInflater.from(parent.context), 16 parent, 17 false 18) 19 return HighlightViewHolder(binding) 20 } 21 22 override fun onBindViewHolder(holder: HighlightViewHolder, position: Int) { 23 holder.bind(song[position]) 24 } 25 26 override fun getItemCount(): Int = song.size 27 28 class HighlightViewHolder(private val binding: ItemHighlightSongBinding) : 29 RecyclerView.ViewHolder(binding.root) { 30 31 fun bind(song: Song) { 32 binding.imageHighlight.setImageResource(song.imageRes) 33 } 34 } 35 }

HomeFragment.kt

Tabel 5 Source Code HomeFragment.kt XML

1	package com.example.modul3xml.ui.home
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.ViewGroup
8	import androidx.fragment.app.Fragment

```

9 import androidx.fragment.app.viewModels
10 import androidx.lifecycle.Lifecycle
11 import androidx.lifecycle.lifecycleScope
12 import androidx.lifecycle.repeatOnLifecycle
13 import androidx.navigation.fragment.findNavController
14 import androidx.recyclerview.widget.LinearLayoutManager
15 import com.example.modul3xml.R
16 import com.example.modul3xml.databinding.FragmentHomeBinding
17 import com.example.modul3xml.viewmodel.SongViewModel
18 import com.example.modul3xml.viewmodel.SongViewModelFactory
19 import kotlinx.coroutines.launch
20 import timber.log.Timber
21
22 class HomeFragment : Fragment() {
23
24     private var _binding: FragmentHomeBinding? = null
25     private val binding get() = _binding!!
26
27     private val viewModel: SongViewModel by viewModels {
28         SongViewModelFactory("Yuukosoo")
29     }
30
31     override fun onCreateView(
32         inflater: LayoutInflater,
33         container: ViewGroup?,
34         savedInstanceState: Bundle?
35     ): android.view.View {
36         _binding = FragmentHomeBinding.inflate(inflater, container, false)
37         return binding.root
38     }
39
40     override fun onViewCreated(view: android.view.View, savedInstanceState: Bundle?) {
41         super.onViewCreated(view, savedInstanceState)
42
43         setupMenu()
44         setupRecyclerViews()
45         observeViewModel()
46     }
47
48     private fun setupMenu() {
49         binding.toolbarHome.setOnMenuItemClickListener { item ->
50             when (item.itemId) {
51                 R.id.menu_settings -> {
52                     findNavController().navigate(R.id.action_homeFragment_to_settingsFragment)
53                         true
54                     }
55                 else -> false
56             }
57         }
58         binding.toolbarHome.title = getString(R.string.app_name)
59     }
60
61     private fun setupRecyclerViews() {
62         binding.rvHighlight.layoutManager =
63             LinearLayoutManager(requireContext(), LinearLayoutManager.HORIZONTAL, false)
64         binding.rvSong.layoutManager = LinearLayoutManager(requireContext())
65     }
66
67     private fun observeViewModel() {
68         viewLifecycleOwner.lifecycleScope.launch {
69             viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {

```

```

69
70         launch {
71             viewModel.songs.collect { songList ->
72                 binding.rvHighlight.adapter = HighlightAdapter(songList)
73
74                 binding.rvSong.adapter = SongListAdapter(
75                     songs = songList,
76                     onBrowserClick = { song ->
77                         viewModel.onBrowserClicked(song.url)
78                     },
79                     onDetailClick = { song ->
80                         viewModel.onSongDetailClicked(song)
81                     }
82                 )
83             }
84         }
85
86         launch {
87             viewModel.selectedSongId.collect { id ->
88                 if (id != null) {
89                     val song = viewModel.songs.value.find { it.id == id
90
91                         Timber.d("Navigasi ke Detail. Lagu dipilih:
92                             ${song?.title}")
93
94                         val bundle = Bundle().apply {
95                             putInt("songId", id)
96                         }
97
98                         findNavController().navigate(R.id.action_homeFragment_to_detailFragment, bundle)
99
100                         viewModel.clearSelection()
101                     }
102                 }
103             }
104
105             launch {
106                 viewModel.navigateToUrl.collect { url ->
107                     if (url != null) {
108                         startActivity(Intent(Intent.ACTION_VIEW,
109                             Uri.parse(url)))
110                         viewModel.clearUrlNavigation()
111                     }
112                 }
113             }
114
115             override fun onDestroyView() {
116                 super.onDestroyView()
117                 _binding = null
118             }
119
120             override fun onResume() {
121                 super.onResume()
122                 _binding?.let { binding ->
123                     binding.toolbarHome.title = getString(R.string.app_name)
124                 }
125         }

```

SongListAdapter.kt

Tabel 6 Source Code SongListAdapter.kt XML

```
1 package com.example.modul3xml.ui.home
2
3 import android.view.LayoutInflater
4 import android.view.ViewGroup
5 import androidx.recyclerview.widget.RecyclerView
6 import com.example.modul3xml.data.Song
7 import com.example.modul3xml.databinding.ItemSongBinding
8 import com.example.modul3xml.util.LanguageSettings
9
10 class SongListAdapter(
11     private val songs: List<Song>,
12     private val onBrowserClick: (Song) -> Unit,
13     private val onDetailClick: (Song) -> Unit
14 ) : RecyclerView.Adapter<SongListAdapter.SongViewHolder>() {
15
16     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
17     SongViewHolder {
18         val binding = ItemSongBinding.inflate(
19             LayoutInflater.from(parent.context),
20             parent,
21             false
22         )
23         return SongViewHolder(binding)
24     }
25
26     override fun onBindViewHolder(holder: SongViewHolder, position: Int) {
27         holder.bind(songs[position], onBrowserClick, onDetailClick)
28     }
29
30     override fun getItemCount(): Int = songs.size
31
32     class SongViewHolder(private val binding: ItemSongBinding) :
33     RecyclerView.ViewHolder(binding.root) {
34
35         fun bind(
36             song: Song,
37             onBrowserClick: (Song) -> Unit,
38             onDetailClick: (Song) -> Unit
39         ) {
40             val isIndonesian =
41             LanguageSettings.isIndonesian(binding.root.context)
42
43             binding.apply {
44                 imageSong.setImageResource(song.imageRes)
45                 textTitle.text = song.title
46                 textYear.text = song.year
47                 textMean.text = if (isIndonesian) song.meanId else song.meanEn
48
49                 buttonBrowser.setOnClickListener { onBrowserClick(song) }
50                 buttonDetail.setOnClickListener { onDetailClick(song) }
```

49	}
50	}
51	}
52	}

SettingsFragment.kt

Tabel 7 Source Code SettingsFragment.kt XML

1	package com.example.modul3xml.ui.settings
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.ViewGroup
6	import androidx.fragment.app.Fragment
7	import androidx.navigation.fragment.findNavController
8	import com.example.modul3xml.R
9	import com.example.modul3xml.databinding.FragmentSettingsBinding
10	import com.example.modul3xml.util.LanguageSettings
11	
12	class SettingsFragment : Fragment() {
13	
14	private var _binding: FragmentSettingsBinding? = null
15	private val binding get() = _binding!!
16	
17	override fun onCreateView(
18	inflater: LayoutInflater,
19	container: ViewGroup?,
20	savedInstanceState: Bundle?
21	): android.view.View {
22	_binding = FragmentSettingsBinding.inflate(inflater, container, false)
23	return binding.root
24	}
25	
26	override fun onViewCreated(view: android.view.View, savedInstanceState: Bundle?) {
27	super.onViewCreated(view, savedInstanceState)
28	
29	setupToolbar()
30	setupLanguageButtons()
31	}
32	
33	private fun setupToolbar() {
34	binding.toolbarSettings.title = getString(R.string.title_settings)
35	binding.toolbarSettings.setNavigationOnClickListener {
36	findNavController().popBackStack()
37	}
38	}
39	
40	private fun setupLanguageButtons() {
41	binding.buttonIndonesia.setOnClickListener {
42	LanguageSettings.setLocale(requireActivity(), "id")
43	refreshTexts()
44	}
45	
46	binding.buttonEnglish.setOnClickListener {
47	LanguageSettings.setLocale(requireActivity(), "en")

48	refreshTexts()
49	}
50	}
51	
52	private fun refreshTexts() {
53	binding.toolbarSettings.title = getString(R.string.title_settings)
54	binding.buttonIndonesia.text = getString(R.string.btn_indonesia)
55	binding.buttonEnglish.text = getString(R.string.btn_english)
56	}
57	
58	override fun onDestroyView() {
59	super.onDestroyView()
60	_binding = null
61	}
62	}

LanguageSettings.kt

Tabel 8 Source Code LanguageSettings.kt XML

1	package com.example.modul3xml.util
2	
3	import android.app.Activity
4	import android.content.Context
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.content.edit
7	import androidx.core.os.LocaleListCompat
8	
9	object LanguageSettings {
10	
11	private const val PREF_NAME = "app_settings"
12	private const val KEY_LANG = "selected_language"
13	
14	fun applySavedLocale(context: Context) {
15	val savedLanguage = getSavedLanguage(context)
16	AppCompatActivity.setApplicationLocales(LocaleListCompat.forLanguageTags(savedLanguage))
17	}
18	
19	fun setLocale(activity: Activity, languageCode: String) {
20	saveLanguage(activity, languageCode)
21	AppCompatActivity.setApplicationLocales(LocaleListCompat.forLanguageTags(languageCode))
22	}
23	
24	fun isIndonesian(context: Context): Boolean {
25	val language = context.resources.configuration.locales[0].language
26	return language == "id" language == "in"
27	}
28	
29	private fun getSavedLanguage(context: Context): String {
30	val prefs = context.getSharedPreferences(PREF_NAME,
31	Context.MODE_PRIVATE)
32	return prefs.getString(KEY_LANG, "id") ?: "id"
33	}
34	
35	private fun saveLanguage(context: Context, languageCode: String) {
	val prefs = context.getSharedPreferences(PREF_NAME,

36	Context.MODE_PRIVATE)
37	prefs.edit {
38	putString(KEY_LANG, languageCode)
39	}
40	}

SongViewModel.kt

Tabel 9 Source Code SongViewModel.kt XML

1	package com.example.modul3xml.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import com.example.modul3xml.data.Song
5	import com.example.modul3xml.data.SongData
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	import kotlinx.coroutines.flow.asStateFlow
9	import timber.log.Timber
10	
11	class SongViewModel(private val username: String) : ViewModel() {
12	private val _songs = MutableStateFlow<List<Song>>(emptyList())
13	val songs: StateFlow<List<Song>> = _songs.asStateFlow()
14	
15	private val _selectedSongId = MutableStateFlow<Int?>(null)
16	val selectedSongId: StateFlow<Int?> = _selectedSongId.asStateFlow()
17	
18	private val _navigateToUrl = MutableStateFlow<String?>(null)
19	val navigateToUrl: StateFlow<String?> = _navigateToUrl.asStateFlow()
20	
21	
22	
23	init {
24	Timber.d("User: \$username - Menginisialisasi ViewModel")
25	
26	_songs.value = SongData.SongList
27	
28	Timber.d("Data lagu berhasil masuk ke dalam list: \${_songs.value} item")
29	}
30	
31	fun onSongDetailClicked(song: Song){
32	Timber.d("Tombol Detail ditekan untuk lagu: \${song.title} (ID: \${song.id})")
33	_selectedSongId.value = song.id
34	}
35	
36	fun clearSelection(){
37	_selectedSongId.value = null
38	}
39	
40	fun onBrowserClicked(url: String){
41	Timber.d("Tombol Explicit Intent ditekan untuk URL: \$url")
42	_navigateToUrl.value = url

43	}
44	
45	fun clearUrlNavigation() {
46	_navigateToUrl.value = null
47	}
48	}

SongViewModelfactory.kt

Tabel 10 SongViewModelFactory.kt XML

1	package com.example.modul3xml.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class SongViewModelFactory(private val username: String) : ViewModelProvider.Factory {
7	@Suppress("UNCHECKED_CAST")
8	override fun <T : ViewModel> create(modelClass: Class<T>): T {
9	if (modelClass.isAssignableFrom(SongViewModel::class.java)) {
10	return SongViewModel(username) as T
11	}
12	throw IllegalArgumentException("Unknown ViewModel class")
13	}
14	}

MainActivity.kt

Tabel 11 Source Code MainActivity.kt XML

1	package com.example.modul3xml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import com.example.modul3xml.databinding.ActivityMainBinding
6	import com.example.modul3xml.util.LanguageSettings
7	
8	class MainActivity : AppCompatActivity() {
9	
10	private lateinit var binding: ActivityMainBinding
11	
12	override fun onCreate(savedInstanceState: Bundle?) {
13	LanguageSettings.applySavedLocale(this)
14	super.onCreate(savedInstanceState)
15	
16	binding = ActivityMainBinding.inflate(layoutInflater)

17	setContentView(binding.root)
18	}
19	}

MyApplication.kt

Tabel 12 Source Code MyApplication.kt XML

1	package com.example.modul3xml
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MyApplication : Application(){
7	override fun onCreate() {
8	super.onCreate()
9	
10	Timber.plant(Timber.DebugTree())
11	
12	}
13	}

activity_main.xml

Table 1 Source Code activity_main.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:background="@color/background_black">
9	<androidx.fragment.app.FragmentContainerView
10	android:id="@+id/nav_host_fragment"
11	android:name="androidx.navigation.fragment.NavHostFragment"
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	app:defaultNavHost="true"
15	app:navGraph="@navigation/nav_graph"
16	tools:layout="@layout/fragment_home" />
17	
18	</FrameLayout>

fragment_detail.xml

Table 2 Source Code fragment_detail.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:fitsSystemWindows="true"
8	android:orientation="vertical"
9	android:background="@color/background_black">
10	<com.google.android.material.appbar.MaterialToolbar
11	android:id="@+id/toolbar_detail"
12	android:layout_width="match_parent"
13	android:layout_height="?attr/actionBarSize"
14	android:paddingTop="8dp"
15	android:background="@color/background_black"
16	app:navigationIcon="@drawable/ic_arrow_back"
17	app:navigationIconTint="@color/white"
18	app:titleTextColor="@color/white" />
19	
20	<ScrollView
21	android:layout_width="match_parent"
22	android:layout_height="match_parent"
23	android:fillViewport="true">
24	
25	<LinearLayout
26	android:layout_width="match_parent"
27	android:layout_height="wrap_content"
28	android:orientation="vertical"
29	android:padding="16dp">
30	
31	<com.google.android.material.imageview.ShapeableImageView
32	android:id="@+id/image_detail"
33	android:layout_width="match_parent"
34	android:layout_height="260dp"
35	android:contentDescription="@string/cd_song_image"
36	android:clipToOutline="true"
37	android:scaleType="centerCrop"
38	app:shapeAppearanceOverlay="@style/RoundedCornerImage" />
39	
40	<LinearLayout
41	android:layout_width="match_parent"
42	android:layout_height="wrap_content"
43	android:layout_marginTop="20dp"
44	android:gravity="center_vertical"
	android:orientation="horizontal">

45	
46	<TextView
47	android:id="@+id/text_detail_title"
48	android:layout_width="0dp"
49	android:layout_height="wrap_content"
50	android:layout_weight="1"
51	android:textColor="@color/white"
52	android:textSize="28sp"
53	android:textStyle="bold" />
54	
55	<TextView
56	android:id="@+id/text_detail_year"
57	android:layout_width="wrap_content"
58	android:layout_height="wrap_content"
59	android:textColor="@color/text_gray"
60	android:textSize="22sp" />
61	
62	</LinearLayout>
63	
64	<TextView
65	android:layout_width="wrap_content"
66	android:layout_height="wrap_content"
67	android:layout_marginTop="8dp"
68	android:text="@string/title_meaning"
69	android:textColor="@color/white"
70	android:textStyle="bold" />
71	
72	<TextView
73	android:id="@+id/text_detail_mean"
74	android:layout_width="match_parent"
75	android:layout_height="wrap_content"
76	android:layout_marginTop="8dp"
77	android:textColor="@color/white" />
78	
79	</LinearLayout>
80	
81	</ScrollView>
82	
83	</LinearLayout>
84	

fragment_home.xml

Table 3 Source Code fragment_home.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout

	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:fitsSystemWindows="true"
7	android:orientation="vertical"
8	android:background="@color/background_black">
9	
10	<com.google.android.material.appbar.MaterialToolbar
11	android:id="@+id/toolbar_home"
12	android:layout_width="match_parent"
13	android:layout_height="?attr/actionBarSize"
14	android:background="@color/background_black"
15	app:titleTextColor="@color/white"
16	app:menu="@menu/menu_home" />
17	
18	<androidx.core.widget.NestedScrollView
19	android:layout_width="match_parent"
20	android:layout_height="match_parent"
21	android:fillViewport="true"
22	android:overScrollMode="never">
23	
24	<LinearLayout
25	android:layout_width="match_parent"
26	android:layout_height="wrap_content"
27	android:orientation="vertical">
28	
29	<TextView
30	android:id="@+id/text_highlight"
31	android:layout_width="match_parent"
32	android:layout_height="wrap_content"
33	android:layout_marginStart="16dp"
34	android:layout_marginTop="16dp"
35	android:layout_marginEnd="16dp"
36	android:text="@string/title_highlight"
37	android:textColor="@color/white"
38	android:textSize="16sp"
39	android:textStyle="bold" />
40	
41	<androidx.recyclerview.widget.RecyclerView
42	android:id="@+id/rv_highlight"
43	android:layout_width="match_parent"
44	android:layout_height="120dp"
45	android:layout_marginTop="12dp"
46	android:clipToPadding="false"
47	android:paddingStart="16dp"
48	android:paddingEnd="16dp"
49	android:overScrollMode="never"
50	android:nestedScrollingEnabled="false" />
51	
52	<androidx.recyclerview.widget.RecyclerView

53	android:id="@+id/rv_games"
54	android:layout_width="match_parent"
55	android:layout_height="wrap_content"
56	android:layout_marginTop="32dp"
57	android:clipToPadding="false"
58	android:paddingBottom="8dp"
59	android:overScrollMode="never"
60	android:nestedScrollingEnabled="false" />
61	
62	</LinearLayout>
63	
64	</androidx.core.widget.NestedScrollView>
65	
66	</LinearLayout>
67	

fragment_settings.xml

Table 4 Source Code fragment_settings.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:fitsSystemWindows="true"
7	android:orientation="vertical"
8	android:background="@color/background_black">
9	
10	<com.google.android.material.appbar.MaterialToolbar
11	android:id="@+id/toolbar_settings"
12	android:layout_width="match_parent"
13	android:layout_height="?attr/actionBarSize"
14	android:background="@color/background_black"
15	app:navigationIcon="@drawable/ic_arrow_back"
16	app:navigationIconTint="@color/white"
17	app:titleTextColor="@color/white" />
18	
19	<LinearLayout
20	android:layout_width="match_parent"
21	android:layout_height="match_parent"
22	android:orientation="vertical"
23	android:gravity="center"
24	android:paddingTop="16dp"
25	android:paddingHorizontal="48dp">
26	
27	<com.google.android.material.button.MaterialButton

28	android:id="@+id/button_indonesia"
29	style="@style/Widget.Material3.Button"
30	android:layout_width="match_parent"
31	android:layout_height="wrap_content"
32	android:text="@string/btn_indonesia"
33	app:backgroundTint="@color/button_blue" />
34	
35	<Space
36	android:layout_width="1dp"
37	android:layout_height="16dp" />
38	
39	<com.google.android.material.button.MaterialButton
40	android:id="@+id/button_english"
41	style="@style/Widget.Material3.Button"
42	android:layout_width="match_parent"
43	android:layout_height="wrap_content"
44	android:text="@string/btn_english"
45	app:backgroundTint="@color/button_blue" />
46	
47	</LinearLayout>
48	
49	</LinearLayout>

item_highlight_song.xml

Table 5 Source Code item_highlight_song.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="200dp"
6	android:layout_height="120dp"
7	android:layout_marginEnd="12dp"
8	app:cardBackgroundColor="@color/card_dark"
9	app:cardCornerRadius="12dp"
10	app:cardElevation="2dp"
11	app:strokeWidth="0dp">
12	
13	<ImageView
14	android:id="@+id/image_highlight"
15	android:layout_width="match_parent"
16	android:layout_height="match_parent"
17	android:scaleType="centerCrop"
18	android:contentDescription="@string/cd_song_image" />
19	
20	</com.google.android.material.card.MaterialCardView>
21	

Item_song.xml

Table 6 Source Code item_song.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="175dp"
7	android:layout_marginStart="16dp"
8	android:layout_marginEnd="16dp"
9	android:layout_marginBottom="12dp"
10	app:cardBackgroundColor="@color/card_dark"
11	app:cardCornerRadius="16dp"
12	app:cardElevation="2dp"
13	app:strokeWidth="0dp">
14	<LinearLayout
15	android:layout_width="match_parent"
16	android:layout_height="match_parent"
17	android:orientation="horizontal"
18	android:padding="12dp">
19	
20	<com.google.android.material.imageview.ShapeableImageView
21	android:id="@+id/image_song"
22	android:layout_width="100dp"
23	android:layout_height="150dp"
24	android:padding="2dp"
25	android:scaleType="centerCrop"
26	app:shapeAppearanceOverlay="@style/RoundedCornerImage" />
27	
28	<LinearLayout
29	android:layout_width="0dp"
30	android:layout_height="match_parent"
31	android:layout_weight="1"
32	android:layout_marginStart="12dp"
33	android:orientation="vertical">
34	
35	<LinearLayout
36	android:layout_width="match_parent"
37	android:layout_height="wrap_content"
38	android:orientation="horizontal"
39	android:gravity="center_vertical">
40	
41	<TextView
42	android:id="@+id/text_title"
43	android:layout_width="0dp"
44	android:layout_height="wrap_content"
45	android:layout_weight="1"


```

46         android:textColor="@color/white"
47         android:textStyle="bold"
48         android:maxLines="1"
49         android:ellipsize="end" />
50
51     <TextView
52         android:id="@+id/text_year"
53         android:layout_width="wrap_content"
54         android:layout_height="wrap_content"
55         android:textColor="@color/text_gray"
56         android:textSize="12sp" />
57
58 </LinearLayout>
59
60 <TextView
61     android:id="@+id/text_mean"
62     android:layout_width="match_parent"
63     android:layout_height="wrap_content"
64     android:layout_marginTop="8dp"
65     android:textColor="@color/text_light_gray"
66     android:textSize="12sp"
67     android:lineSpacingExtra="2dp"
68     android:maxLines="2"
69     android:ellipsize="end" />
70
71 <Space
72     android:layout_width="match_parent"
73     android:layout_height="0dp"
74     android:layout_weight="1" />
75
76 <LinearLayout
77     android:layout_width="match_parent"
78     android:layout_height="wrap_content"
79     android:gravity="bottom|end"
80     android:orientation="horizontal">
81
82     <com.google.android.material.button.MaterialButton
83         android:id="@+id/button_browser"
84         style="@style/Widget.Material3.Button"
85         android:layout_width="wrap_content"
86         android:layout_height="36dp"
87         android:insetTop="0dp"
88         android:insetBottom="0dp"
89         android:minWidth="0dp"
90         android:text="@string/btn_browser"
91         android:textSize="11sp"
92         app:backgroundTint="@color/button_blue" />
93     <Space
94         android:layout_width="8dp"
95         android:layout_height="1dp" />
96

```

97	<code><com.google.android.material.button.MaterialButton</code>
98	<code> android:id="@+id/button_detail"</code>
99	<code> style="@style/Widget.Material3.Button"</code>
100	<code> android:layout_width="wrap_content"</code>
101	<code> android:layout_height="36dp"</code>
102	<code> android:insetTop="0dp"</code>
103	<code> android:insetBottom="0dp"</code>
104	<code> android:minWidth="0dp"</code>
105	<code> android:text="@string/btn_detail"</code>
106	<code> android:textSize="11sp"</code>
107	<code> app:backgroundTint="@color/button_blue" /></code>
108	
109	<code></LinearLayout></code>
110	<code></LinearLayout></code>
111	
112	<code></LinearLayout></code>
113	
114	<code></com.google.android.material.card.MaterialCardView></code>
115	
116	

menu_home.xml

Table 7 Source Code menu_home.xml XML Soal 1

1	<code><?xml version="1.0" encoding="utf-8"?></code>
2	<code><menu</code>
	<code> xmlns:android="http://schemas.android.com/apk/res/android"</code>
3	<code> xmlns:app="http://schemas.android.com/apk/res-auto"></code>
4	
5	<code> <item</code>
6	<code> android:id="@+id/menu_settings"</code>
7	<code> android:icon="@drawable/ic_settings"</code>
8	<code> android:title="@string/title_settings"</code>
9	<code> app:showAsAction="ifRoom" /></code>
10	<code></menu></code>

COMPOSE

Song.kt

Tabel 13. Source Code Song.kt

1	package com.example.modul4compose.model
2	
3	data class Song(
4	val id: Int,
5	val title: String,
6	val year: String,
7	val meanId: String,
8	val meanEn: String,
9	val imageRes: Int,
10	val url: String
11	)

SongData.kt

Tabel 14. Source Code SongData.kt

1	package com.example.modul4compose.data
2	
3	import com.example.modul4compose.R
4	import com.example.modul4compose.model.Song
5	
6	object SongData {
7	val songLists = listOf(
8	Song(
9	id = 1,
10	title = "Scott Street",
11	year = "2017",
12	meanId = "Lagu emosional tentang nostalgia dan perasaan terasing dari
13	Phoebe Bridgers.",
14	meanEn = "An emotional song about nostalgia and feeling alienated by
15	Phoebe Bridgers.",
16	imageRes = R.drawable.scootstreet,
17	url =
18	"https://open.spotify.com/track/2luFPefbgeR3QLVJWATlrr?si=ab2c7ad661ad4168"
19	),
20	Song(
21	id = 2,
22	title = "The Night We Met",
23	year = "2015",
24	meanId = "Balada melankolis dari Lord Huron tentang kerinduan untuk
25	kembali ke masa lalu.",
26	meanEn = "A melancholy ballad by Lord Huron about longing to return
27	to the past.",
28	imageRes = R.drawable.thenightwemet,
29	url =
30	"https://open.spotify.com/track/3hRV0jL3vUpRrcy398teAU?si=bfd097b1bc824f91"
31	),
32	Song(
33	id = 3,
34	title = "Hello",
35	year = "2015",
36	meanId = "Karya musik folk minimalis dari Ichiko Aoba dengan petikan
37	gitar yang lembut.",

31	meanEn = "A folk rock song by Ichiko Aoba with a soft guitar.",
32	imageRes = R.drawable.hello,
33	url =
	"https://open.spotify.com/track/795bHfsDk7EC34IsLqaXfp?si=0a8eba42ee67448d"
34	),
35	Song(
36	id = 4,
37	title = "Light Song",
38	year = "2016",
39	meanId = "Karya musik yang menenangkan dengan harmoni vokal yang
	indah.",
40	meanEn = "A soothing musical piece featuring beautiful vocal
	harmonies.",
41	imageRes = R.drawable.lightson,
42	url =
	"https://open.spotify.com/track/4KXZ4MJKT0RYGhijfQ08Jw?si=9e08213f078c4a8f"
43	),
44	Song(
45	id = 5,
46	title = "Childhood",
47	year = "1995",
48	meanId = "Lagu dari daniel.mp3 yang sangat personal tentang masa lalu
	saat kecil.",
49	meanEn = "A personal song about the past when I was a child.",
50	imageRes = R.drawable.childhood,
51	url =
	"https://open.spotify.com/track/06dDLXf4gPbE02gvH8E6G9?si=1d289d479e104445"
52	)
53	)
54	}

AppNavigation.kt

Tabel 15. Source Code AppNavigation.kt

1	package com.example.modul4compose.ui.navigation
2	
3	import androidx.compose.runtime.Composable
4	import androidx.lifecycle.viewmodel.compose.viewModel
5	import androidx.navigation.compose.NavHost
6	import androidx.navigation.compose.composable
7	import androidx.navigation.compose.rememberNavController
8	import com.example.modul4compose.ui.screen.DetailScreen
9	import com.example.modul4compose.ui.screen.HomeScreen
10	import com.example.modul4compose.ui.screen.SettingsScreen
11	import com.example.modul4compose.viewmodel.SongViewModel
12	import com.example.modul4compose.viewmodel.SongViewModelFactory
13	
14	@Composable
15	fun AppNavigation() {
16	val navController = rememberNavController()
17	
18	
19	val viewModel: SongViewModel = viewModel(
20	factory = SongViewModelFactory("Yuuukosoo")
21	)
22	
23	NavHost(navController = navController, startDestination = "home") {
24	composable("home") {
25	

26	HomeScreen(navController = navController, viewModel = viewModel)
27	}
28	composable("detail/{songId}") { backStackEntry ->
29	val songId =
	backStackEntry.arguments?.getString("songId")?.toIntOrNull()
30	
31	DetailScreen(navController = navController, songId = songId,
	viewModel = viewModel)
32	}
33	composable("settings") {
34	SettingsScreen(navController = navController)
35	}
36	}
37	}

DetailScreen.kt

Tabel 16. Source Code DetailScreen.kt

1	package com.example.modul4compose.ui.screen
2	
3	import androidx.compose.foundation.*
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material.icons.Icons
7	import androidx.compose.material.icons.automirrored.filled.ArrowBack
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.*
10	import androidx.compose.ui.*
11	import androidx.compose.ui.draw.clip
12	import androidx.compose.ui.graphics.Color
13	import androidx.compose.ui.layout.ContentScale
14	import androidx.compose.ui.platform.LocalConfiguration
15	import androidx.compose.ui.res.painterResource
16	import androidx.compose.ui.res.stringResource
17	import androidx.compose.ui.text.font.FontWeight
18	import androidx.compose.ui.unit.dp
19	import androidx.navigation.NavController
20	import com.example.modul4compose.viewmodel.SongViewModel
21	import timber.log.Timber
22	
23	@Composable
24	fun DetailScreen(navController: NavController, songId: Int?, viewModel:
	SongViewModel) {
25	val songList by viewModel.songs.collectAsState()
26	val song = songList.find { it.id == songId }
27	val lang = LocalConfiguration.current.locales.get(0).language
28	val isId = lang == "in" lang == "id"
29	
30	LaunchedEffect(song) {
31	song?.let {
32	Timber.d("Menampilkan Detail untuk lagu: \${it.title}")
33	}
34	}
35	
36	Scaffold(
37	topBar = {
38	Surface(color = Color.Black, shadowElevation = 4.dp) {
39	Row(Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal

```

= 8.dp), verticalAlignment = Alignment.CenterVertically) {
40     IconButton(onClick = { navController.popBackStack() }) {
41         Icon(Icons.AutoMirrored.Filled.ArrowBack, null, tint =
Color.White)
42     }
43     Text(song?.title ?: "Detail", style =
MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold, modifier =
Modifier.padding(start = 8.dp))
44     }
45     }
46     }
47     ) { p ->
48         song?.let {

49         Column(Modifier.fillMaxSize().padding(p).verticalScroll(rememberScrollState()).p
adding(16.dp)) {
50             Image(painterResource(it.imageRes), null, modifier =
Modifier.fillMaxWidth().height(260.dp).clip(RoundedCornerShape(16.dp)),
contentScale = ContentScale.Crop)
51             Row(Modifier.fillMaxWidth().padding(vertical = 20.dp),
horizontalArrangement = Arrangement.SpaceBetween) {
52                 Text(it.title, style =
MaterialTheme.typography.headlineMedium, fontWeight = FontWeight.Bold, modifier
= Modifier.weight(1f))
53                 Text(it.year, style = MaterialTheme.typography.titleLarge,
color = Color.Gray)
54             }

55             Text(stringResource(com.example.modul4compose.R.string.titlemean), fontWeight =
FontWeight.Bold)
56             Text(if (isId) it.meanId else it.meanEn, modifier =
Modifier.padding(top = 8.dp))
57         }
58     }
59 }
60 }

```

HomeScreen.kt

Tabel 17. Source Code DetailScreen.kt

```

1 package com.example.modul4compose.ui.screen
2
3 import android.content.Intent
4 import androidx.compose.foundation.Image
5 import androidx.compose.foundation.layout.*
6 import androidx.compose.foundation.lazy.*
7 import androidx.compose.foundation.shape.RoundedCornerShape
8 import androidx.compose.material.icons.Icons
9 import androidx.compose.material.icons.filled.Settings
10 import androidx.compose.material3.*
11 import androidx.compose.runtime.*
12 import androidx.compose.ui.*
13 import androidx.compose.ui.draw.clip
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.layout.ContentScale
16 import androidx.compose.ui.platform.LocalContext
17 import androidx.compose.ui.res.painterResource
18 import androidx.compose.ui.res.stringResource
19 import androidx.compose.ui.text.font.FontWeight

```

```

20 import androidx.compose.ui.text.style.TextOverflow
21 import androidx.compose.ui.unit.*
22 import androidx.navigation.NavController
23 import com.example.modul4compose.model.Song
24 import com.example.modul4compose.viewmodel.SongViewModel
25 import com.example.modul4compose.R
26
27
28 @Composable
29 fun HomeScreen(navController: NavController, viewModel: SongViewModel) {
30     val songList by viewModel.songs.collectAsState()
31     val navigateToUrl by viewModel.navigateToUrl.collectAsState()
32     val context = LocalContext.current
33
34     LaunchedEffect(navigateToUrl) {
35         navigateToUrl?.let { url ->
36             val intent = Intent(Intent.ACTION_VIEW, android.net.Uri.parse(url))
37             context.startActivity(intent)
38             viewModel.clearUrlNavigation()
39         }
40     }
41
42     Scaffold(
43         topBar = {
44             Surface(color = Color.Black, shadowElevation = 4.dp) {
45                 Row(
46                     modifier =
47                     Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal =
48                     16.dp),
49                     verticalAlignment = Alignment.CenterVertically
50                     ) {
51                         Text(stringResource(R.string.app_name), style =
52                         MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold, modifier =
53                         Modifier.weight(1f))
54                         IconButton(onClick = { navController.navigate("settings") }) {
55                             Icon(Icons.Default.Settings, null, tint = Color.White)
56                         }
57                     }
58                 ) { padding ->
59                     LazyColumn(modifier = Modifier.padding(padding)) {
60                         item {
61                             Text(stringResource(R.string.title_highlight), modifier =
62                             Modifier.padding(16.dp), fontWeight = FontWeight.SemiBold)
63                             LazyRow(contentPadding = PaddingValues(horizontal = 16.dp),
64                             horizontalArrangement = Arrangement.spacedBy(12.dp)) {
65                                 items(songList) { song ->
66                                     Card(Modifier.size(200.dp, 120.dp), shape =
67                                     RoundedCornerShape(12.dp)) {
68                                         Image(painterResource(song.imageRes), null,
69                                         contentScale = ContentScale.Crop, modifier = Modifier.fillMaxSize())
70                                     }
71                                 }
72                                 Spacer(Modifier.height(24.dp))
73                             }
74                         items(songList) { song ->
75                             SongListItem(
76                                 song = song,
77                                 onBrowserClick = { viewModel.onBrowserClicked(song.url) },
78                                 onDetailClick = {

```

```

74         viewModel.onDetailClicked(song)
75         navController.navigate("detail/${song.id}")
76     }
77 )
78 }
79 }
80 }
81 }
82
83 @Composable
84 fun SongListItem(song: Song, onBrowserClick: () -> Unit, onDetailClick: () ->
Unit) {
85     Card(
86         modifier = Modifier.padding(horizontal = 16.dp, vertical =
8.dp).fillMaxWidth().height(175.dp),
87         shape = RoundedCornerShape(16.dp),
88         colors = CardDefaults.cardColors(containerColor = Color(0xFF1A1A1A))
89     ) {
90         Row(Modifier.padding(12.dp).fillMaxSize()) {
91             Image(
92                 painterResource(song.imageRes), null,
93                 modifier = Modifier.size(100.dp,
150.dp).clip(RoundedCornerShape(12.dp)),
94                 contentScale = ContentScale.Crop
95             )
96             Column(Modifier.padding(start = 12.dp).fillMaxHeight()) {
97                 Text(song.title, fontWeight = FontWeight.Bold, maxLines = 1,
overflow = TextOverflow.Ellipsis)
98                 Text(song.meanEn, style = MaterialTheme.typography.bodySmall,
color = Color.LightGray, maxLines = 2)
99                 Spacer(Modifier.weight(1f))
100                Row(Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.End) {
101                    val blue = ButtonDefaults.buttonColors(containerColor =
Color(0xFF2196F3))
102                    Button(onClick = onBrowserClick, colors = blue, modifier =
Modifier.height(36.dp)) {
103                        Text(stringResource(R.string.btn_browser), fontSize =
11.sp)
104                    }
105                    Spacer(Modifier.width(8.dp))
106                    Button(onClick = onDetailClick, colors = blue, modifier =
Modifier.height(36.dp)) {
107                        Text(stringResource(R.string.btn_detail), fontSize =
11.sp)
108                    }
109                }
110            }
111        }
112    }
113 }

```


SettingScreen.kt

Tabel 18. Source Code SettingScreen.kt

```

1 package com.example.modul4compose.ui.screen
2
3 import android.app.Activity
4 import android.content.res.Configuration
5 import androidx.activity.compose.LocalActivity
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.material.icons.Icons
8 import androidx.compose.material.icons.automirrored.filled.ArrowBack
9 import androidx.compose.material3.*
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.*
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.res.stringResource
14 import androidx.compose.ui.text.font.FontWeight
15 import androidx.compose.ui.unit.dp
16 import java.util.Locale
17 import com.example.modul4compose.R
18
19 @Composable
20 fun SettingsScreen(navController: androidx.navigation.NavController) {
21     val act = LocalActivity.current
22     val blue = ButtonDefaults.buttonColors(containerColor = Color(0xFF2196F3))
23
24     Scaffold(
25         topBar = {
26             Surface(color = Color.Black, shadowElevation = 4.dp) {
27                 Row(
28                     modifier =
29 Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal =
30 8.dp),
31                     verticalAlignment = Alignment.CenterVertically
32                 ) {
33                     IconButton(onClick = { navController.popBackStack() }) {
34                         Icon(Icons.AutoMirrored.Filled.ArrowBack, null, tint =
35 Color.White)
36                     }
37                     Text(stringResource(R.string.title_settings), style =
38 MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold)
39                 }
40             }
41         }
42     ) { p ->
43         Column(
44             modifier = Modifier.fillMaxSize().padding(p),
45             horizontalAlignment = Alignment.CenterHorizontally,
46             verticalArrangement = Arrangement.Center
47         ) {
48             LanguageButton(stringResource(R.string.btn_indonesia), blue) {
49 act?.let { updateLang(it, "id") } }
50             Spacer(Modifier.height(16.dp))
51             LanguageButton(stringResource(R.string.btn_english), blue) { act?.let
52 { updateLang(it, "en") } }
53         }
54     }
55 }
56
57 @Composable
58 fun LanguageButton(text: String, colors: ButtonColors, onClick: () -> Unit) {
59     Button(

```

54	onClick = onClick,
55	modifier = Modifier.fillMaxWidth().padding(horizontal = 48.dp),
56	colors = colors
57	) {
58	Text(text)
59	}
60	}
61	
62	@Suppress("DEPRECATION")
63	private fun updateLang(act: Activity, code: String) {
64	val locale = Locale(code)
65	Locale.setDefault(locale)
66	val config = Configuration(act.resources.configuration)
67	config.setLocale(locale)
68	act.resources.updateConfiguration(config, act.resources.displayMetrics)
69	act.recreate()
70	}

Theme.kt

Tabel 19. Source Code Theme.kt

1	package com.example.modul4compose.ui.theme
2	
3	import androidx.compose.material3.MaterialTheme
4	import androidx.compose.material3.darkColorScheme
5	import androidx.compose.runtime.Composable
6	import androidx.compose.ui.graphics.Color
7	
8	private val DarkColorScheme = darkColorScheme(
9	primary = Color.White,
10	background = Color.Black,
11	surface = Color(0xFF121212),
12	onBackground = Color.White,
13	onSurface = Color.White
14	)
15	
16	@Composable
17	fun Modul3Theme(content: @Composable () -> Unit) {
18	MaterialTheme(
19	colorScheme = DarkColorScheme,
20	content = content
21	)
22	}

SongViewModel.kt

Tabel 20. Source Code SongViewModel.kt

1	package com.example.modul4compose.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import com.example.modul4compose.data.SongData
5	import com.example.modul4compose.model.Song
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	import kotlinx.coroutines.flow.asStateFlow
9	import timber.log.Timber
10	
11	class SongViewModel(private val username: String) : ViewModel() {
12	
13	private val _songs = MutableStateFlow<List<Song>>(emptyList())
14	val songs: StateFlow<List<Song>> = _songs.asStateFlow()
15	
16	private val _navigateToUrl = MutableStateFlow<String?>(null)
17	val navigateToUrl: StateFlow<String?> = _navigateToUrl.asStateFlow()
18	
19	init {
20	Timber.d("User: \$username - Menginisialisasi ViewModel")
21	_songs.value = SongData.songLists
22	Timber.d("Data lagu berhasil masuk ke dalam list: \${_songs.value.size}
23	item")
24	}
25	
26	fun onBrowserClicked(url: String) {
27	Timber.d("Tombol Explicit Intent ditekan untuk URL: \$url")
28	_navigateToUrl.value = url
29	}
30	
31	fun clearUrlNavigation() {
32	_navigateToUrl.value = null
33	}
34	
35	fun onDetailClicked(song: Song) {
36	Timber.d("Tombol Detail ditekan untuk lagu: \${song.title} (ID: \${song.id}
37	Link: \${song.url} Makna: \${song.meanId})")
	}
	}

SongViewModelFactory.kt

Tabel 21. Source Code SongViewModelFactory.kt

1	package com.example.modul4compose.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class SongViewModelFactory(private val username: String) :
	ViewModelProvider.Factory {
7	@Suppress("UNCHECKED_CAST")
8	override fun <T : ViewModel> create(modelClass: Class<T>): T {
9	if (modelClass.isAssignableFrom(SongViewModel::class.java)) {
10	return SongViewModel(username) as T

11	}
12	throw IllegalArgumentException("Unknown ViewModel class")
13	}
14	}

MainActivity.kt

Tabel 22. Source Code MainActivity.kt

1	package com.example.modul4compose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.fillMaxSize
7	import androidx.compose.material3.MaterialTheme
8	import androidx.compose.material3.Surface
9	import androidx.compose.material3.darkColorScheme
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.graphics.Color
12	import com.example.modul4compose.ui.navigation.AppNavigation
13	
14	class MainActivity : ComponentActivity() {
15	override fun onCreate(savedInstanceState: Bundle?) {
16	super.onCreate(savedInstanceState)
17	setContent {
18	MaterialTheme(
19	colorScheme = darkColorScheme(
20	background = Color.Black,
21	surface = Color(0xFF121212),
22	onBackground = Color.White,
23	onSurface = Color.White
24	)
25	) {
26	Surface(
27	modifier = Modifier.fillMaxSize(),
28	color = MaterialTheme.colorScheme.background
29	) {
30	AppNavigation()
31	}
32	}
33	}
34	}
35	}

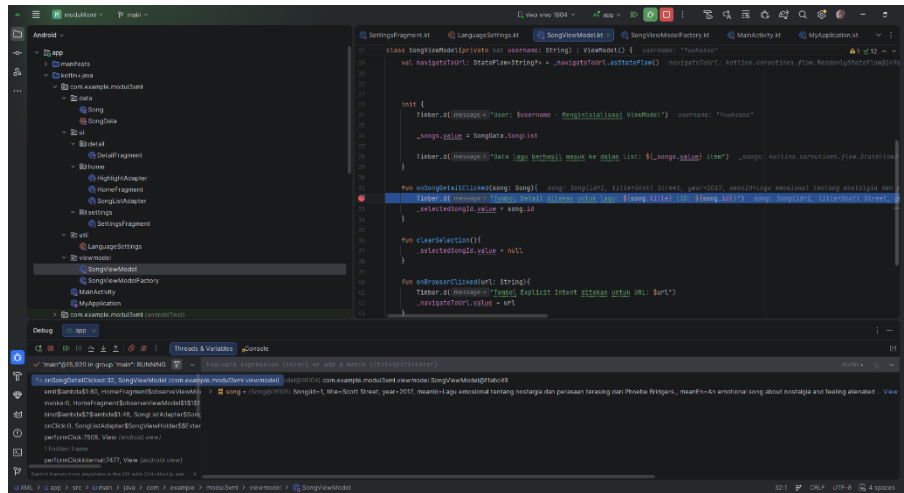
MyApplication.kt

Tabel 23. Source Code MainActivity.kt

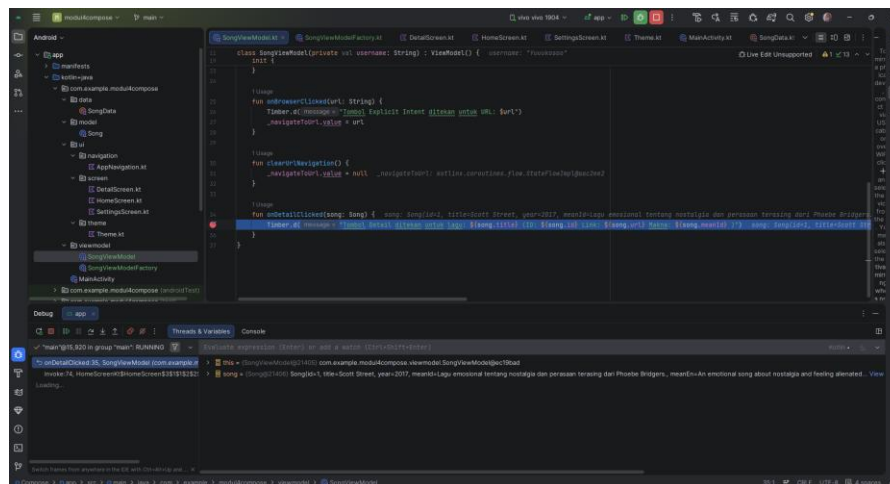
1	package com.example.modul4compose
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MyApplication : Application() {
7	override fun onCreate() {

8	super.onCreate()
9	
10	
11	Timber.plant(Timber.DebugTree())
12	}
13	}

B. Output Program



Gambar 1. Screenshot Hasil Test Debugging Soal 1 XML



Gambar 2 Screenshot Hasil TestDebugging Soal 1 Compose

C. Pembahasan

XML

Song.kt

Pada kode Song.kt merupakan data class yang digunakan sebagai model atau wadah data yang tujuannya untuk membungkus informasi pada lagu dalam objek tunggal.

SongData.kt

Kode ini fungsinya sebagai sumber data yang dimana terdapat variabel SongList menyimpan daftar informasi lagu menggunakan struktur data class Song, mencakup detail judul, tahun rilis, deskripsi dalam dua bahasa, gambar dan tautan musik yang akan digunakan pada ui nantinya pada fungsi findSongById digunakan untuk menelusuri lagu ditentukan berdasarkan ID-nya.

DetailFragment.kt

Pada bagian awal, **DetailFragment** melakukan inisialisasi **SongViewModel** melalui delegasi **by viewModels** dengan menyertakan **SongViewModelFactory**. Langkah teknis ini memastikan bahwa Fragment terhubung dengan penyedia data yang sama dan dapat menerima parameter "Yuukosoo" saat ViewModel diciptakan. Bagian **onViewCreated** tetap menjadi titik awal setelah tampilan diciptakan untuk memicu pengaturan navigasi pada toolbar dan proses pengisian data lagu ke layar. Di dalam fungsi **setupToolbar**, sistem mengatur agar tombol navigasi pada toolbar menjalankan instruksi **popBackStack**, yang secara teknis akan menghapus layar detail dari tumpukan navigasi dan mengembalikan pengguna ke halaman sebelumnya. Perubahan signifikan terdapat pada fungsi **bindSongData**. Berbeda dengan modul sebelumnya, pengambilan data lagu kini dilakukan secara lebih aman melalui **viewModel.songs.value.find**. Di mana data tidak boleh dikelola langsung oleh Fragment, melainkan harus melalui ViewModel. Di sini juga disematkan fitur debugging menggunakan **Timber**; jika data lagu tidak ditemukan berdasarkan ID yang dikirim melalui *arguments*, sistem akan mencatat peringatan **Timber.w**, namun jika berhasil, sistem akan mencatat log **Timber**. Logika pengisian tampilan tetap mempertahankan fitur multibahasa melalui **LanguageSettings**. Jika sistem mendeteksi bahasa Indonesia, maka teks

deskripsi akan diambil dari **song.meanId**, dan jika bahasa Inggris, maka diambil dari **song.meanEn**. Seluruh informasi seperti gambar, judul, tahun, dan deskripsi kemudian dimasukkan ke dalam elemen visual melalui objek **binding**. Terakhir, pada **onDestroyView**, variabel binding diatur menjadi *null* untuk memastikan pembersihan memori

HomeFragment.kt

Pada bagian awal, **HomeFragment** dideklarasikan dengan menginisialisasi **SongViewModel** menggunakan delegasi **by viewModels** dan **SongViewModelFactory**. Fragment mendapatkan *instance* ViewModel yang sama dan telah terisi parameter identitas pengguna sejak awal. Di dalam metode **onViewCreated**, sistem menyiapkan antarmuka pengguna melalui tiga tahapan utama, mengatur menu pada toolbar, mengonfigurasi manajer tata letak (*layout manager*) untuk dua buah RecyclerView, dan memulai proses pemantauan data melalui fungsi **observeViewModel**.

Fungsi **setupMenu** bertugas untuk mendengarkan interaksi pada toolbar; jika pengguna menekan ikon pengaturan, sistem akan menggunakan **findNavController** untuk melakukan navigasi menuju **SettingsFragment**. Sementara itu, fungsi **setupRecyclerViews** kini hanya bertugas menyiapkan kerangka daftar lagu dalam orientasi horizontal untuk *highlight* dan vertikal untuk daftar utama, tanpa mengisi data secara langsung di awal. Inti pada class ini pada fungsi **observeViewModel**. Dengan memanfaatkan **viewLifecycleOwner.lifecycleScope** dan **repeatOnLifecycle(Lifecycle.State.STARTED)**, Fragment akan mendapatkan data dari ViewModel secara aman, di mana proses ini hanya berjalan saat aplikasi sedang aktif di layar pengguna. Terdapat tiga hal utama yang pertama, pemantauan terhadap **viewModel.songs** yang akan memperbarui isi adapter RecyclerView secara otomatis setiap kali ada perubahan data list. Kedua, pemantauan terhadap **viewModel.selectedSongId**; jika nilai ID terpilih tidak kosong, sistem akan mencatat log navigasi menggunakan **Timber.d**, membungkus ID tersebut ke dalam **Bundle**, lalu berpindah ke **DetailFragment**. Ketiga, pemantauan terhadap **viewModel.navigateToUrl** untuk menangani navigasi eksternal. Jika terdapat URL yang dikirim oleh ViewModel, Fragment akan menjalankan **Explicit Intent** untuk membuka web

MyApplication.kt

Pada bagian awal, kelas **MyApplication** ini dideklarasikan sebagai turunan dari kelas **Application**. Method `onCreate` di class ini sebagai titik masuk pertama saat aplikasi diluncurkan, method ini dijalankan oleh sistem Android sebelum activity atau layar pertama muncul pada pengguna. Di dalam method tersebut terdapat **super.onCreate()** yang berfungsi untuk memastikan bahwa seluruh rangkaian inisialisasi dasar dari sistem Android tetap berjalan sebagaimana mestinya. Inti utama dari kelas ini adalah baris kode **Timber.plant**, yang merupakan instruksi untuk menanamkan atau mengaktifkan pustaka logging **Timber**. **Timber.DebugTree()** memberikan instruksi kepada sistem untuk mencatat setiap aktivitas log ke dalam jendela Logcat secara otomatis selama aplikasi berada dalam mode pengembangan (debug).

SongViewModel.kt

Pada bagian awal, class **SongViewModel** dideklarasikan dengan mewariskan class **ViewModel** serta menerima parameter `username` melalui konstruktor. **ViewModel** ini bertugas sebagai pusat penyimpanan data dan logika bisnis yang tetap terjaga meskipun terjadi perubahan konfigurasi pada perangkat, seperti rotasi layar. Inti dari kelas ini adalah pengelolaan data secara reaktif menggunakan pola **Backing Property**, di mana variabel yang diawali garis bawah seperti **_songs**, **_selectedSongId**, dan **_navigateToUrl** didefinisikan sebagai **MutableStateFlow** yang bersifat privat agar hanya bisa dimodifikasi dari dalam **ViewModel**. Sebaliknya, variabel tanpa garis bawah seperti **songs**, **selectedSongId**, dan **navigateToUrl** diekspos sebagai **StateFlow** yang bersifat *read-only* agar lapisan antarmuka (UI) hanya bisa mengamati perubahan data tanpa bisa mengubah isinya secara langsung.

Pada blok **init**, sistem secara otomatis menjalankan rangkaian perintah awal saat **ViewModel** pertama kali dibuat, dimulai dengan mencetak log identitas pengguna menggunakan **Timber**, kemudian memindahkan daftar lagu dari **SongData.SongList** ke dalam data **_songs**, dan diakhiri dengan mencatat keberhasilan masuknya data tersebut ke dalam logcat. Fungsi **onSongDetailClicked** dan **onBrowserClicked** berperan sebagai jembatan komunikasi dari UI. Ketika berinteraksi. Terakhir, terdapat fungsi **clearSelection** dan **clearUrlNavigation** mengosongkan kembali nilai *state* menjadi *null* setelah proses navigasi selesai

SongViewModelFactory.kt

SongViewModelFactory ini dideklarasikan dengan mengimplementasikan antarmuka **ViewModelProvider.Factory**. sebagai penyedia atau instruksi khusus bagi sistem Android untuk menciptakan *instance* dari **ViewModel** yang memiliki parameter *Inti* dari class ini terletak pada metode **create**, yang merupakan fungsi yang secara otomatis dipanggil oleh sistem saat sebuah **ViewModel** diminta oleh **Activity** atau **Fragment**. Di dalam metode tersebut, sistem melakukan pengecekan logika menggunakan fungsi **isAssignableFrom** untuk memastikan bahwa kelas **ViewModel** yang sedang diminta oleh aplikasi memang benar merupakan **SongViewModel**. Jika kondisi tersebut terpenuhi, maka **Factory** akan secara manual menciptakan objek **SongViewModel** baru sembari menyisipkan data **username** ke dalamnya, lalu mengembalikannya sebagai tipe generik **T**. Penggunaan anotasi **@Suppress("UNCHECKED_CAST")** di sini diperlukan untuk memberitahu *compiler* agar mengabaikan peringatan keamanan tipe data saat melakukan proses konversi paksa (*casting*) objek menjadi tipe yang diinginkan. Sebaliknya, jika kelas yang diminta tidak dikenal atau tidak sesuai dengan peruntukannya, sistem akan melemparkan pesan kesalahan berupa **IllegalArgumentException**

HighlightAdapter.kt

Bagian **onCreateViewHolder** mengambil desain dari file **XML** yaitu **item_highlight_song** kemudian merakit ke sebuah objek **view**, kemudian pada bagian **onBindViewHolder** ini digunakan untuk memasukkan data lagu ke dalam wadah sesuai dengan urutan. Bagian **getItemCount** digunakan untuk memberitahu seberapa banyak lagu yang ada di dalam daftar. Dan terakhir bagian **HighlightViewHolder** digunakan untuk menandai elemen gambar lagu

SongListAdapter.kt

Pada bagian awal, **Adapter** ini menerima tiga parameter utama yaitu daftar lagu serta dua fungsi klik **onBrowserClick** dan **onDetailClick** yang berfungsi sebagai instruksi aksi ketika tombol di dalam item ditekan. Bagian **onCreateViewHolder** digunakan untuk menyiapkan desain setiap baris lagu. Di sini, **ItemSongBinding** digunakan untuk membaca file desain **item_song.xml** dan menyimpannya dalam variabel **binding**. Bagian **onBindViewHolder** bertugas untuk

menghubungkan data lagu dari urutan tertentu ke dalam tampilan, sedangkan getItemCount berfungsi untuk memberitahu sistem jumlah total lagu yang akan ditampilkan dalam daftar. Bagian SongViewHolder, khususnya pada fungsi bind, merupakan inti dari pengisian data ke elemen visual. Pertama, sistem melakukan pengecekan bahasa melalui LanguageSettings. Jika aplikasi menggunakan bahasa Indonesia, maka teks arti lagu akan mengambil data dari song.meanId, namun jika tidak, maka akan mengambil dari song.MeanEn. Di dalam blok binding.apply, semua elemen desain seperti gambar lagu, judul, dan tahun rilis diisi secara otomatis sesuai dengan data yang ada. Terakhir, bagian ini juga mengatur dua aksi klik: jika buttonBrowser diklik, maka akan menjalankan fungsi untuk membuka browser, dan jika buttonDetail diklik, maka akan menjalankan fungsi untuk menampilkan detail lagu tersebut.

SettingsFragment.kt

Pada bagian awal ada inisialisasi Binding dengan dua variabel yaitu _binding dan binding yang digunakan untuk mengakses elemen desain pada XML. Lalu onCreateView digunakan untuk membaca desain fragment_settings.xml dan disimpan di variabel binding agar komponen UI di dalamnya bisa dikendalikan. Lalu bagian onViewCreated, program memanggil fungsi-fungsi secara berurutan untuk menyiapkan tampilan. Bagian setupToolbar berfungsi untuk mengatur bar navigasi di bagian atas. Di sini aplikasi menentukan judul halaman pengaturan dan memberikan logika pada tombol navigasi kembali; jika ditekan, maka fungsi popBackStack() akan dijalankan untuk mengembalikan pengguna ke halaman sebelumnya (Home). Bagian setupLanguageButtons mengatur logika pada dua tombol pilihan bahasa. Jika buttonIndonesia ditekan, maka aplikasi akan mengatur bahasa sistem menjadi Indonesia melalui LanguageSettings. Begitu juga jika buttonEnglish ditekan, maka bahasa akan berubah menjadi Inggris. Setelah bahasa dipilih, fungsi refreshTexts akan langsung dipanggil. Bagian refreshTexts berfungsi untuk memperbarui teks pada layar secara *real-time*. Fungsi ini mengambil ulang string dari sumber daya aplikasi (R.string) untuk judul toolbar dan teks pada tombol. Dan terakhir onDestroyView, fungsinya untuk membersihkan memori dengan mengubah variabel _binding menjadi null saat screen atau fragment ini ditutup.

LanguageSettings.kt

Pada bagian awal, kelas ini didefinisikan sebagai object LanguageSettings, yang bertindak sebagai

kelas pembantu (*utility*) untuk mengatur lokalisasi atau bahasa di dalam aplikasi. Kelas ini menggunakan `SharedPreferences` dengan identitas `PREF_NAME` dan `KEY_LANG` untuk menyimpan pilihan bahasa pengguna secara permanen di memori perangkat. Bagian `applySavedLocale` berfungsi untuk memuat bahasa yang telah disimpan sebelumnya dan menerapkannya ke seluruh aplikasi. Bagian `setLocale` digunakan saat pengguna menekan tombol ganti bahasa di halaman pengaturan. Di sini, sistem akan melakukan dua hal sekaligus: menyimpan kode bahasa baru (seperti "id" atau "en") ke memori perangkat melalui fungsi `saveLanguage`, dan secara otomatis memperbarui konfigurasi bahasa aplikasi secara *real-time* menggunakan `AppCompatDelegate`. Bagian `isIndonesian` merupakan fungsi pengecekan untuk mengetahui apakah bahasa yang sedang aktif saat ini adalah bahasa Indonesia. Fungsi ini bekerja dengan memeriksa konfigurasi sistem, dan hasilnya (benar atau salah) digunakan oleh bagian lain aplikasi, seperti pada `Adapter lagu`, untuk menentukan apakah akan menampilkan teks dalam bahasa Indonesia atau Inggris. Terakhir, bagian `getSavedLanguage` dan `saveLanguage` adalah fungsi internal yang bekerja di balik layar. Fungsi-fungsi ini bertugas untuk membaca data dari memori dan menuliskan perubahan data ke dalam pengaturan aplikasi, sehingga pilihan bahasa tetap tersimpan meskipun aplikasi dikeluarkan

MainActivity.kt

Pada bagian awal terdapat inisialisasi variabel binding menggunakan `ActivityMainBinding`. Variabel ini berfungsi sebagai jembatan untuk mengakses seluruh elemen desain yang ada di dalam file `activity_main.xml` secara langsung tanpa perlu menggunakan `findViewById`. Di dalam fungsi `onCreate`, hal pertama yang dilakukan sebelum menampilkan antarmuka adalah memanggil `'LanguageSettings.applySavedLocale(this)'`. Bagian ini berfungsi untuk memastikan bahwa aplikasi langsung menggunakan bahasa yang telah dipilih dan disimpan oleh pengguna (Indonesia atau Inggris) tepat saat aplikasi baru saja dibuka. Setelah pengaturan bahasa diterapkan, sistem menjalankan perintah `'Activity MainBinding.inflate(layoutInflater)'` untuk membaca file desain utama. Hasil pembacaan desain tersebut kemudian disimpan ke dalam variabel binding. Terakhir, fungsi `'setContentView(binding.root)'` digunakan untuk menampilkan seluruh komponen visual dari file XML ke layar ponsel pengguna. Karena kelas ini adalah `MainActivity`, maka ia bertugas

sebagai kontainer atau wadah utama yang mengatur navigasi dan tempat munculnya fragment-fragment lain di dalam aplikasi.

activity_main.xml

Pada bagian awal, desain ini menggunakan `FrameLayout` sebagai kontainer utama yang mencakup seluruh layar aplikasi. Bagian ini memiliki warna latar belakang `@color/background_`. Di dalam `FrameLayout` tersebut terdapat elemen `FragmentContainerView` yang berfungsi sebagai "panggung" atau wadah tempat ditampilkan berbagai halaman aplikasi secara bergantian. Dengan komponen ini, aplikasi tidak perlu berpindah `Activity`, melainkan cukup menukar konten `Fragment` di dalam satu wadah saja. Atribut `android:name` menetapkan penggunaan `NavHostFragment`, yang merupakan pengatur utama sistem navigasi `Jetpack`. Bagian `app:navGraph` menghubungkan desain ini dengan file alur navigasi (`nav_graph`), yang menentukan urutan halaman dan tujuan perpindahan layar. Bagian `app:defaultNavHost="true"` memastikan bahwa sistem navigasi ini terintegrasi dengan tombol "kembali" (back button) pada ponsel, sehingga ketika pengguna menekan tombol back, aplikasi akan otomatis kembali ke halaman sebelumnya sesuai dengan riwayat navigasi. Terakhir, atribut `tools:layout="@layout/fragment_home"` berfungsi sebagai alat bantu di dalam `Android Studio`. Gunanya agar pada saat proses pengembangan, kita bisa melihat pratinjau atau simulasi tampilan `fragment_home` langsung di dalam editor, meskipun atribut ini tidak akan memberikan pengaruh apa pun saat aplikasi dijalankan di perangkat asli.

fragment_detail.xml

Pada bagian awal, desain ini menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal dan latar belakang berwarna hitam. Atribut `fitsSystemWindows="true"` ditambahkan agar konten aplikasi tidak tertutup. Bagian `MaterialToolbar` dengan ID `toolbar_detail` berfungsi sebagai bar navigasi atas. Di sini terdapat sebuah `navigationIcon` berupa panah kembali agar pengguna bisa dengan mudah keluar dari halaman detail. Warna ikon dan judul diatur menjadi putih agar kontras dengan latar belakang hitam yang digunakan. Lalu terdapat `ScrollView` yang membungkus area konten utama. Bagian

ini sangat penting agar pengguna bisa menggeser layar ke bawah (scroll) jika informasi lagu yang ditampilkan cukup panjang dan melebihi tinggi layar. Atribut `fillViewport="true"` memastikan konten tetap mengisi ruang layar. Di dalam area konten, terdapat `ShapeableImageView` dengan ID `image_detail` untuk menampilkan gambar lagu dalam ukuran besar. Penggunaan `shapeAppearanceOverlay` memberikan efek sudut melengkung pada gambar. Bagian `LinearLayout` horizontal di bawah gambar digunakan untuk menata judul lagu dan tahun rilis secara berdampingan. Judul lagu memiliki `layout_weight="1"` agar ia mengambil ruang paling luas di sisi kiri, sedangkan tahun rilis diletakkan di sisi kanan dengan ukuran teks yang sedikit lebih kecil dan warna abu-abu. Terakhir, terdapat label statis bertuliskan "Meaning" yang diambil dari file string sebagai judul informasi, serta sebuah `TextView` dengan ID `text_detail_mean` di bagian paling bawah

fragment_home.xml

Pada bagian awal, desain ini menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal dan latar belakang berwarna hitam. Atribut `fitsSystemWindows="true"` digunakan agar tata letak aplikasi tidak berbenturan dengan area status bar di bagian atas layar.

Bagian `MaterialToolbar` dengan ID `toolbar_home` berfungsi sebagai bar navigasi utama. Di sini aplikasi menampilkan judul halaman dengan warna teks putih dan menghubungkan file menu (`@menu/menu_home`) yang berisi ikon pengaturan bahasa di pojok kanan atas.

Lalu terdapat `NestedScrollView` yang membungkus seluruh konten halaman. Penggunaan komponen ini memungkinkan pengguna untuk menggeser (scroll) seluruh isi halaman. Atribut `overScrollMode="never"` digunakan untuk menghilangkan efek pantulan saat digeser sampai ujung.

Di dalam konten utama, terdapat sebuah `TextView` yang berfungsi sebagai label judul untuk bagian sorotan (highlight). Teksnya diambil dari sumber daya string (`@string/title_highlight`) dengan gaya huruf tebal dan warna putih.

Bagian `RecyclerView` dengan ID `rv_highlight` digunakan untuk menampilkan daftar lagu pilihan dalam ukuran kecil (tinggi 120dp). Dengan pengaturan yang tepat di bagian kode program, daftar

ini akan ditampilkan secara mendatar (horizontal) sehingga gambar-gambar lagu bisa digeser ke samping.

Terakhir, terdapat RecyclerView kedua dengan ID `rv_song` yang terletak di bagian bawah. Daftar ini digunakan untuk menampilkan list lagu utama yang disusun memanjang ke bawah (vertikal). Atribut `nestedScrollingEnabled="false"` dipasang pada kedua daftar tersebut agar proses penggeseran layar dikendalikan sepenuhnya oleh NestedScrollView induk, sehingga tidak terjadi konflik saat pengguna mengusap layar.

fragemnt_settings.xml

Pada bagian awal, desain ini menggunakan LinearLayout sebagai kontainer utama dengan orientasi vertikal dan latar belakang berwarna hitam. Atribut `fitsSystemWindows="true"` digunakan agar tata letak aplikasi tidak berbenturan dengan area status bar di bagian atas layar.

Bagian MaterialToolbar dengan ID `toolbar_settings` berfungsi sebagai bar navigasi atas pada halaman pengaturan. Di sini disediakan navigationIcon berupa panah kembali dengan warna putih agar pengguna bisa menavigasi balik ke layar sebelumnya.

Di bawah toolbar, terdapat LinearLayout kedua yang berfungsi sebagai wadah utama tombol pilihan bahasa. Wadah ini menggunakan atribut `gravity="center"` agar semua tombol yang ada di dalamnya berada tepat di tengah-tengah layar, serta diberikan jarak (padding) agar posisi tombol tidak terlalu menempel ke pinggir layar.

Bagian MaterialButton dengan ID `button_indonesia` dan `button_english` digunakan sebagai tombol pilihan bahasa dengan warna biru yang diambil dari `@color/button_blue`

Terakhir, di antara kedua tombol tersebut terdapat elemen Space setinggi 16dp. Bagian ini berfungsi untuk memberikan jarak atau ruang kosong yang pas antara tombol bahasa Indonesia dan bahasa Inggris.

item_highlight_song.xml

Pada bagian awal, desain ini menggunakan MaterialCardView sebagai kontainer utama yang berfungsi sebagai bingkai untuk setiap item lagu pilihan. Bingkai ini memiliki ukuran lebar 200dp dan tinggi 120dp, serta diberikan jarak di sisi kanan (layout_marginEnd) sebesar 12dp.

Bagian kartu ini memiliki sudut melengkung sebesar 12dp dan warna latar belakang gelap (@color/card_dark). Terdapat juga sedikit efek bayangan atau elevation sebesar 2dp.

Di dalam kartu tersebut, terdapat sebuah ImageView dengan ID image_highlight yang ukurannya memenuhi seluruh isi kartu. Penggunaan atribut scaleType="centerCrop" sangat penting di sini untuk memastikan bahwa gambar lagu yang ditampilkan akan terpotong secara proporsional dan memenuhi seluruh ruang kartu tanpa terlihat gepeng. Terakhir, atribut contentDescription ditambahkan untuk mendukung fitur aksesibilitas, memberikan deskripsi pada gambar

item_song.xml

Pada bagian awal, desain ini menggunakan MaterialCardView sebagai wadah utama untuk setiap item lagu dalam daftar. Kartu ini memiliki sudut yang melengkung sebesar 16dp dengan warna latar belakang gelap (@color/card_dark). Di dalamnya terdapat LinearLayout horizontal yang membagi tampilan menjadi dua

bagian: gambar di sisi kiri dan informasi teks di sisi kanan. Bagian gambar menggunakan ShapeableImageView dengan ID image_song yang memiliki ukuran tetap 100x150dp dan sudut melengkung agar serasi dengan bentuk kartu.

Lalu terdapat LinearLayout vertikal di sisi kanan yang menampung seluruh informasi lagu. Di bagian paling atas, terdapat susunan horizontal untuk judul lagu (text_title) dan tahun rilis (text_year). Judul lagu dibuat tebal dan memiliki fitur ellipsis="end", sehingga jika judulnya terlalu panjang, teks akan otomatis terpotong dengan tanda titik-titik. Di bawah judul, terdapat TextView dengan ID text_mean untuk menampilkan cuplikan makna lagu. Teks ini dibatasi maksimal dua baris dan menggunakan warna abu-abu. Terdapat elemen Space dengan berat (layout_weight="1") yang berfungsi sebagai pendorong otomatis agar bagian tombol selalu berada

di posisi paling bawah Terakhir, di sudut kanan bawah terdapat dua MaterialButton, yaitu button_browser dan button_detail. Kedua tombol ini menggunakan gaya Material 3 dengan ukuran yang lebih ringkas (tinggi 36dp) dan warna biru, memberikan opsi kepada pengguna

untuk langsung membuka link musik atau masuk ke halaman detail lagu tersebut.

menu_home.xml

Pada bagian awal, kode ini merupakan file konfigurasi Menu yang digunakan untuk menambahkan aksi atau tombol pada bagian Toolbar.

Di dalamnya terdapat sebuah item dengan ID menu_settings. Item ini menggunakan ikon yang diambil dari (@drawable/ic_settings) dan memiliki judul yang merujuk pada string title_settings

Bagian app:showAsAction="ifRoom" adalah atribut yang memerintahkan sistem agar menampilkan tombol tersebut langsung sebagai ikon di bilah toolbar jika masih ada ruang yang tersedia. Namun, jika layar ponsel terlalu sempit, maka tombol tersebut akan otomatis disembunyikan ke dalam menu titik tiga

Terakhir, menu ini nantinya akan dihubungkan dengan kode program (Fragment atau Activity) agar ketika ikon pengaturan tersebut ditekan, aplikasi dapat menjalankan perintah untuk berpindah ke halaman pengaturan.

COMPOSE

SongData.kt

Pada bagian awal, kode ini didefinisikan sebagai object SongData, yang berfungsi sebagai pusat penyimpanan data atau sumber informasi lagu yang akan digunakan di seluruh aplikasi. Dengan menggunakan bentuk object, data ini bersifat statis dan dapat langsung dipanggil tanpa perlu membuat objek baru.

Di dalamnya terdapat sebuah variabel bernama songLists yang berisi sebuah daftar (listOf) dari beberapa objek Song. Daftar ini menjadi kumpulan data utama yang akan ditampilkan pada layar aplikasi, baik dalam bentuk daftar vertikal maupun sorotan (*highlight*).

Setiap lagu di dalam daftar ini telah diisi dengan id, title (judul), dan year (tahun rilis).

Untuk mendukung fitur multibahasa, setiap entri lagu menyertakan penjelasan dalam dua versi, yaitu meanId untuk bahasa Indonesia dan MeanEn untuk bahasa Inggris. Selain itu, bagian imageRes menghubungkan lagu dengan file gambar yang ada di folder *drawable* melalui referensi R.drawable.

Terakhir, setiap lagu juga dilengkapi dengan url yang merujuk pada link streaming musik. Data inilah yang nantinya dikirim ke komponen UI (Compose) untuk diproses, sehingga dapat melihat gambar, membaca makna lagu, hingga membuka link tersebut.

Song.kt

Pada bagian awal, kode ini mendefinisikan sebuah data class bernama Song yang berfungsi sebagai model data atau wadah informasi untuk setiap lagu di dalam aplikasi.

Di dalamnya terdapat beberapa variabel utama: id sebagai identitas unik setiap lagu, title untuk menyimpan judul lagu, dan year untuk menyimpan tahun rilisnya. Data-data ini nantinya akan dipanggil oleh komponen UI untuk ditampilkan.

Untuk mendukung fitur multibahasa, kelas ini menyediakan dua variabel teks, yaitu meanId untuk penjelasan makna lagu dalam bahasa Indonesia dan meanEn untuk penjelasan dalam bahasa

Inggris. Pemisahan ini memudahkan aplikasi saat harus memilih teks mana yang akan muncul berdasarkan pengaturan bahasa yang dipilih.

Bagian `imageRes` menggunakan tipe data integer karena berfungsi untuk menyimpan ID referensi gambar yang ada di folder *drawable* aplikasi. Terakhir, terdapat variabel url yang menyimpan alamat link internet, yang akan digunakan ketika menekan tombol untuk membuka lagu tersebut melalui browser.

AppNavigation.kt

Pada bagian awal, fungsi Composable **AppNavigation** ini bertugas sebagai pusat kendali navigasi atau perpindahan layar di dalam aplikasi. Di sini dibuat variabel **navController** menggunakan **rememberNavController()** yang berfungsi untuk melacak riwayat kunjungan layar dan mengelola perintah perpindahan antar halaman. Penambahan krusial pada Modul 4 ini adalah inisialisasi variabel **viewModel** menggunakan fungsi **viewModel()** yang dipadukan dengan **SongViewModelFactory**. Bagian ini berfungsi untuk menciptakan *instance* **SongViewModel** sekaligus menyuntikkan parameter string berupa username "**Yuuukosoo**" ke dalamnya, sehingga data tersebut dapat dikelola secara konsisten selama aplikasi berjalan.

Bagian **NavHost** digunakan sebagai wadah utama yang mengatur alur navigasi dengan **startDestination = "home"**. Ini artinya, saat aplikasi pertama kali dibuka, sistem akan secara otomatis mengarahkan pengguna untuk melihat layar utama terlebih dahulu.

Di dalam **NavHost**, terdapat beberapa rute yang didefinisikan menggunakan fungsi **composable**. Rute pertama adalah "**home**" yang akan menampilkan **HomeScreen**. Berbeda dengan sebelumnya, di sini variabel **navController** dan **viewModel** ikut dikirimkan sebagai parameter. Tujuannya agar halaman utama tidak hanya memiliki kemampuan navigasi, tetapi juga dapat mengakses serta memantau aliran data lagu secara reaktif dari **ViewModel**.

Bagian rute kedua adalah "**detail/{songId}**" yang digunakan untuk menuju halaman detail secara dinamis. Rute ini membawa parameter berupa ID lagu yang dipilih pengguna. Di dalamnya, sistem akan mengambil nilai **songId** dari **backStackEntry** dan mengubahnya menjadi tipe data angka (Integer). Data ID tersebut kemudian dikirimkan ke **DetailScreen** bersama dengan

objek **viewModel** agar informasi lagu yang ditampilkan tetap sinkron dengan data yang ada di pusat penyimpanan **ViewModel**.

Terakhir, terdapat rute "**settings**" yang akan menampilkan **SettingsScreen**. Sama seperti rute lainnya, halaman ini juga menerima **NavController** agar pengguna bisa melakukan aksi navigasi, seperti menekan tombol kembali untuk menuju halaman sebelumnya secara lancar.

DetailScreen.kt

Pada bagian awal, fungsi Composable **DetailScreen** ini menerima tiga parameter penting, yaitu **NavController** untuk mengelola navigasi, **songId** untuk menentukan lagu mana yang harus ditampilkan, serta **viewModel** sebagai penyedia data utama. Berbeda dengan sebelumnya, sistem kini mengambil daftar lagu secara reaktif dari **ViewModel** menggunakan **viewModel.songs.collectAsState()**. Dari aliran data tersebut, sistem mencari data lagu yang cocok berdasarkan ID serta melakukan pengecekan bahasa sistem melalui **LocalConfiguration** untuk menentukan penggunaan bahasa Indonesia atau Inggris. Selain itu, terdapat penambahan baris **LaunchedEffect(song)** yang berfungsi untuk mencatat log menggunakan **Timber.d** ketika halaman detail berhasil dimuat, guna memantau data lagu yang sedang ditampilkan secara *real-time*.

Bagian **Scaffold** digunakan sebagai kerangka utama layar yang menyediakan struktur dasar. Di dalam atribut **topBar**, terdapat **Surface** berwarna hitam yang berisi sebuah **IconButton** dengan ikon panah kembali; jika ditekan, fungsi **popBackStack()** akan dijalankan untuk mengembalikan pengguna ke halaman sebelumnya. Di samping tombol tersebut, judul lagu ditampilkan secara dinamis sesuai data dari **ViewModel** dengan gaya huruf tebal.

Bagian konten utama dibungkus menggunakan **Column** yang dilengkapi dengan fungsi **verticalScroll**. Hal ini bertujuan agar seluruh informasi lagu dapat digeser ke bawah (*scroll*) dengan lancar jika isinya melebihi tinggi layar ponsel. Seluruh konten diberikan jarak (*padding*) agar tidak terlalu menempel ke pinggir layar.

Di dalam kolom tersebut, komponen **Image** diletakkan paling atas untuk menampilkan poster lagu. Gambar ini memiliki tinggi 260dp dan menggunakan **RoundedCornerShape(16.dp)** untuk

memberikan efek sudut melengkung yang estetik, serta **ContentScale.Crop** agar gambar memenuhi ruang yang tersedia tanpa merusak proporsinya.

Bagian informasi teks diatur menggunakan **Row** untuk menyusun judul lagu dan tahun rilis secara berdampingan. Judul lagu diberikan bobot (*weight*) agar mengambil ruang lebih luas, sementara tahun rilis diletakkan di sisi kanan dengan warna abu-abu.

HomeScreen.kt

Pada bagian awal, fungsi Composable **HomeScreen** menggunakan **Scaffold** sebagai kerangka utama layar. Penambahan utama pada Modul 4 ini adalah integrasi **viewModel** sebagai sumber data tunggal. Di sini, variabel **songList** dan **navigateToUrl** didefinisikan menggunakan fungsi **collectAsState()** untuk mengambil data dari aliran *StateFlow* secara reaktif. Terdapat pula blok **LaunchedEffect** yang bertugas memantau status navigasi; jika **ViewModel** memperbarui nilai URL, sistem akan secara otomatis menjalankan **Explicit Intent** untuk membuka peramban dan kemudian mengosongkan kembali *state* tersebut melalui **viewModel.clearUrlNavigation()**. Di bagian **topBar**, terdapat **Surface** berwarna hitam yang menampilkan nama aplikasi dan sebuah **IconButton** menuju rute "settings".

Bagian konten utama diatur menggunakan **LazyColumn** agar daftar konten dapat di-*scroll* secara vertikal. Berbeda dengan modul sebelumnya, data yang ditampilkan pada bagian **Highlight** dan daftar utama kini diambil langsung dari variabel **songList** yang dikelola oleh **ViewModel**, bukan lagi dari objek statis. Hal ini memastikan tampilan selalu sinkron dengan data terbaru yang ada di dalam memori **ViewModel**.

Di bawah bagian sorotan, setiap data lagu diproses melalui komponen **SongListItem**. Pada modul ini, komponen tersebut tidak lagi menjalankan logika navigasi secara mandiri, melainkan menerima dua fungsi lambda yaitu **onBrowserClick** dan **onDetailClick**. Saat pengguna berinteraksi, komponen ini akan mengirimkan sinyal ke **ViewModel** melalui fungsi **viewModel.onBrowserClicked** atau **viewModel.onDetailClicked** sebelum akhirnya berpindah layar.

Komponen **SongListItem** sendiri mendefinisikan tampilan setiap baris lagu di dalam sebuah **Card** berwarna gelap dengan sudut melengkung 16dp. Sebuah **Row** digunakan untuk menyusun gambar lagu di sisi kiri dan kolom informasi di sisi kanan secara berdampingan. Untuk bagian teks informasi, judul dan makna lagu tetap dilengkapi dengan fitur **Ellipsis**

Terakhir, di sudut kanan bawah setiap kartu lagu terdapat dua tombol berwarna biru. Tombol **Browser** berfungsi untuk memicu proses pembukaan URL lagu, sementara tombol **Detail** akan menjalankan instruksi untuk mencatat aktivitas klik pada log **Timber** di ViewModel sekaligus membawa pengguna masuk ke halaman **DetailScreen** dengan membawa ID lagu terkait.

SettingScreen.kt

Pada bagian awal, fungsi Composable SettingsScreen ini menginisialisasi variabel act untuk mendapatkan referensi Activity yang sedang berjalan serta menentukan skema warna biru yang akan digunakan secara seragam pada tombol-tombol di halaman ini.

Bagian Scaffold digunakan sebagai kerangka utama layar. Di dalam atribut topBar, terdapat sebuah Surface berwarna hitam yang menampung IconButton dengan ikon panah kembali. Jika tombol ini ditekan, fungsi navController.popBackStack() akan dijalankan untuk membawa pengguna kembali ke layar sebelumnya. Judul halaman diambil secara dinamis dari sumber daya string agar mendukung multibahasa.

Bagian konten utama diatur menggunakan Column yang memiliki pengaturan Arrangement.Center dan Alignment.CenterHorizontally. Hal ini bertujuan agar tombol-tombol pilihan bahasa muncul tepat di tengah-tengah layar ponsel. Di dalamnya terdapat dua pemanggilan fungsi LanguageButton yang dipisahkan oleh sebuah Spacer setinggi 16dp sebagai ruang kosong antar tombol.

Fungsi LanguageButton sendiri adalah sebuah komponen untuk membuat tombol dengan desain yang konsisten. Tombol ini diatur agar memenuhi lebar layar dengan jarak (padding) horizontal sebesar 48dp. Saat tombol ditekan, ia akan mengirimkan instruksi ke fungsi updateLang dengan membawa kode bahasa yang sesuai, yaitu "id" untuk Indonesia atau "en" untuk Inggris.

Terakhir, terdapat fungsi `updateLang` yang menangani logika teknis penggantian bahasa. Fungsi ini bekerja dengan cara membuat objek `Locale` baru, memperbarui konfigurasi sumber daya (*resources*) pada sistem Android, dan yang paling penting adalah menjalankan perintah `act.recreate()`. Perintah ini akan memulai ulang aktivitas agar seluruh tampilan aplikasi disegarkan secara otomatis menggunakan bahasa yang baru saja dipilih oleh pengguna.

Theme.kt

Pada bagian awal, kode ini mendefinisikan sebuah skema warna yang disimpan dalam variabel `DarkColorScheme`. Di sini, aplikasi dikonfigurasi untuk menggunakan tema gelap (*Dark Mode*) dengan menetapkan warna hitam (`Color.Black`) sebagai latar belakang utama dan warna putih sebagai warna primer untuk elemen-elemen penting.

Di dalam skema tersebut, terdapat pengaturan warna `surface` dengan kode hex `0xFF121212`) yang digunakan sebagai warna latar belakang untuk komponen seperti kartu (*Card*) atau bilah atas (*Toolbar*). Selain itu, warna `onBackground` dan `onSurface` diatur menjadi putih agar semua teks yang muncul di atas latar belakang gelap tersebut tetap kontras dan mudah dibaca.

Lalu terdapat fungsi `Composable` utama bernama `Modul3Theme`. Fungsi ini bertindak sebagai pembungkus (*wrapper*) atau penyedia tema bagi seluruh konten aplikasi. Dengan menggunakan `MaterialTheme`, semua komponen Jetpack Compose yang berada di dalam lingkup fungsi ini akan secara otomatis mengikuti aturan warna yang telah didefinisikan dalam `DarkColorScheme`.

Terakhir, parameter `content` memungkinkan fungsi ini menerima blok kode UI lainnya. Hal ini memastikan bahwa seluruh layar aplikasi, mulai dari halaman utama hingga halaman pengaturan, memiliki visual yang konsisten.

SongViewModel.kt

Pada bagian awal, kelas **`SongViewModel`** dideklarasikan dengan mewarisi kelas **`ViewModel`** serta menerima parameter **`username`** melalui konstruktornya, yang memungkinkan data identitas pengguna dikirim langsung dari `Factory` saat inisialisasi. `ViewModel` ini berperan sebagai "otak" aplikasi yang bertugas menyimpan data serta logika

navigasi agar tetap bertahan meskipun terjadi perubahan konfigurasi pada perangkat. Inti dari pengelolaan data di kelas ini adalah penggunaan **StateFlow** yang bersifat reaktif dengan pola **Backing Property**. Variabel **_songs** dan **_navigateToUrl** didefinisikan sebagai **MutableStateFlow** yang bersifat privat agar data hanya bisa dimodifikasi dari dalam ViewModel. Sementara itu, variabel **songs** dan **navigateToUrl** diekspos sebagai **StateFlow** (*read-only*) menggunakan fungsi **asStateFlow()**, sehingga lapisan antarmuka (UI) dapat berlangganan pada perubahan data tersebut tanpa memiliki akses untuk mengubah isinya secara langsung.

Pada blok **init**, sistem secara otomatis menjalankan rangkaian perintah awal ketika ViewModel diciptakan, dimulai dengan mencetak log identitas pengguna menggunakan **Timber.d**, kemudian memindahkan daftar lagu dari **SongData.songLists** ke dalam aliran data **_songs**, dan diakhiri dengan mencatat log keberhasilan pemuatan data beserta jumlah itemnya. Kelas ini menyediakan tiga fungsi utama untuk menangani interaksi pengguna. Fungsi **onBrowserClicked** bertugas mencatat log aktivitas tombol *Explicit Intent* dan memperbarui state URL agar UI dapat memicu pembukaan peramban web. Fungsi **clearUrlNavigation** memiliki peran teknis krusial untuk mengosongkan kembali nilai state menjadi null. Terakhir, fungsi **onDetailClicked** berfungsi sebagai penangkap event klik tombol detail, di mana fungsi ini akan mencatat informasi lagu ke dalam log Timber mulai dari judul, ID, tautan, hingga maknanya.

SongViewModelFactory.kt

Pada bagian awal, kelas **SongViewModelFactory** ini dideklarasikan dengan mengimplementasikan antarmuka **ViewModelProvider.Factory**. Kelas ini berfungsi sebagai instruksi khusus atau "pabrik" bagi sistem Android untuk menciptakan *instance* ViewModel yang membutuhkan parameter kustom pada konstruktornya. Secara teknis, sistem Android secara default hanya mengetahui cara membuat ViewModel dengan konstruktor kosong; oleh karena itu, Factory ini diperlukan untuk menyuntikkan data **username** bertipe String ke dalam **SongViewModel**. Inti dari class ini terletak pada metode **create**, yang merupakan fungsi yang di-*override* untuk menentukan bagaimana sebuah ViewModel harus diciptakan. Di dalam metode tersebut, sistem menggunakan teknik *reflection* melalui fungsi **isAssignableFrom** untuk memverifikasi apakah kelas ViewModel yang diminta oleh aplikasi memang benar merupakan tipe **SongViewModel**. Jika pemeriksaan logika tersebut bernilai benar, maka Factory akan

menginstansiasi **SongViewModel** secara manual sembari menyertakan argumen **username**, lalu mengembalikannya sebagai tipe generik **T**. Penggunaan anotasi **@SuppressWarnings("UNCHECKED_CAST")** di sini bertujuan untuk membungkam peringatan dari *compiler* terkait keamanan tipe data saat melakukan konversi objek secara paksa (*casting*). Sebaliknya, jika sistem meminta kelas **ViewModel** yang tidak terdaftar dalam logika **Factory** ini, maka sebuah **IllegalArgumentException** akan dilemparkan untuk memberikan informasi bahwa kelas tersebut tidak dikenali

MainActivity.kt

Pada bagian awal, kelas **MainActivity** ini merupakan titik masuk utama aplikasi yang mewarisi sifat dari **ComponentActivity**. Di dalam fungsi **onCreate**, sistem akan menyiapkan segala kebutuhan aplikasi tepat saat aplikasi mulai dijalankan di ponsel pengguna.

Bagian **setContent** digunakan untuk mendefinisikan seluruh antarmuka pengguna menggunakan **Jetpack Compose**. Di sini, aplikasi tidak lagi menggunakan file **XML**, melainkan langsung menyusun komponen **UI** menggunakan kode **Kotlin**.

Terdapat pengaturan **MaterialTheme** yang secara khusus mendefinisikan skema warna aplikasi melalui **darkColorScheme**. Bagian ini memastikan aplikasi tampil dengan tema gelap, di mana latar belakang diatur menjadi hitam, permukaan komponen diatur menjadi abu-abu sangat gelap, serta semua teks yang muncul di atasnya diatur berwarna putih agar tetap kontras dan mudah dibaca.

Di dalam tema tersebut, terdapat komponen **Surface** yang berfungsi sebagai wadah atau "lantai" paling dasar dari aplikasi. Dengan menggunakan atribut **Modifier.fillMaxSize()**, **Surface** ini akan membentang luas hingga memenuhi seluruh ukuran layar ponsel pengguna dan menerapkan warna latar belakang sesuai tema yang telah dibuat.

Terakhir, di dalam wadah **Surface** tersebut, aplikasi memanggil fungsi **AppNavigation()**. Bagian inilah yang menjadi perintah penutup untuk memulai sistem navigasi, sehingga aplikasi dapat menampilkan layar pertama (**Home**) dan mengelola proses perpindahan antar halaman sesuai dengan rute yang telah ditentukan sebelumnya.

MyApplication.kt

Pada bagian awal, kelas **MyApplication** dideklarasikan sebagai turunan dari kelas **Application**, yang menjadikannya sebagai titik masuk utama dan komponen global yang berjalan sebelum layar mana pun muncul. Metode **onCreate** di sini berfungsi untuk menjalankan inisialisasi awal saat aplikasi pertama kali diluncurkan oleh sistem. Inti dari kelas ini terletak pada baris kode **Timber.plant(Timber.DebugTree())**, yang secara teknis berfungsi untuk mengaktifkan pustaka logging **Timber**. Dengan menggunakan **DebugTree**, sistem akan secara otomatis mengelola pencatatan log pada jendela Logcat

Timber Logging

Pada Modul 4 ini, implementasi **Timber Logging** bertindak sebagai mekanisme pemantauan aplikasi yang lebih canggih dan terstruktur dibandingkan pustaka log standar Android. Proses inisialisasi dilakukan pada level global melalui kelas **MyApplication**, di mana baris kode **Timber.plant(Timber.DebugTree())** berfungsi untuk "menanamkan" sistem pencatatan jejak kodingan agar aktif di seluruh bagian proyek. Secara teknis, penggunaan Timber memberikan efisiensi yang signifikan karena pengembang tidak perlu lagi menentukan label TAG secara manual di setiap baris pemanggilan log; Timber secara cerdas akan mendeteksi nama file kodingan di mana log tersebut dipanggil dan menjadikannya sebagai identitas TAG secara otomatis.

Penerapan log ini disematkan pada beberapa titik strategis untuk memenuhi kriteria pemantauan data dan interaksi pengguna. Pertama, log digunakan di dalam blok **init** pada ViewModel untuk memvalidasi alur data saat daftar lagu berhasil masuk ke dalam aliran *StateFlow*. Kedua, Timber bertugas mencatat setiap interaksi pengguna secara spesifik, seperti saat tombol **Detail** atau tombol **Explicit Intent** untuk membuka peramban ditekan; hal ini bertujuan untuk memastikan bahwa setiap perintah navigasi telah dipicu dengan parameter data yang benar. Ketiga, pada layar detail, log digunakan untuk memaparkan informasi lengkap dari data yang dipilih sesaat setelah proses perpindahan halaman berhasil dilakukan. Melalui jendela **Logcat** di Android Studio, seluruh rangkaian aktivitas ini dapat dipantau secara nyata (*real-time*), sehingga mempermudah pengembang dalam memverifikasi integritas data yang mengalir dari pusat penyimpanan ke antarmuka pengguna serta mempercepat proses identifikasi masalah selama fase pengembangan aplikasi.

Debugger

Secara teknis, **Debugger** adalah alat bantu pengembangan (*development tool*) yang berfungsi untuk menghentikan eksekusi aplikasi secara sementara pada titik tertentu agar pengembang dapat memeriksa kondisi internal sistem secara mendalam. Berbeda dengan *logging* yang hanya menampilkan teks statis, Debugger memungkinkan pemantauan variabel secara nyata (*real-time*), memeriksa penggunaan memori, serta melacak alur logika program baris-per-baris. Fungsi utamanya adalah untuk mengidentifikasi penyebab kesalahan logika atau *bug* yang sulit dideteksi hanya dengan membaca kode sumber.

Proses debugging dimulai dengan menetapkan satu atau lebih **Breakpoint**, yaitu tanda berhenti berupa titik merah yang diletakkan di sebelah kiri nomor baris kodingan yang ingin diamati. Setelah *breakpoint* terpasang, aplikasi dijalankan bukan dengan tombol "Run" biasa, melainkan melalui mode **Debug** (ikon serangga hijau) di Android Studio. Ketika aplikasi mencapai baris yang memiliki *breakpoint*, sistem akan membekukan (*freeze*) sementara jalannya aplikasi.

Fitur Step Over untuk menjalankan baris kode yang sedang ditunjuk saat ini secara utuh dan langsung berpindah ke baris berikutnya di bawahnya dalam lingkup file yang sama. Secara teknis, jika baris tersebut memanggil sebuah fungsi (seperti fungsi log atau fungsi navigasi), **Step Over** akan mengeksekusi fungsi tersebut di balik layar tanpa membawa jendela editor masuk ke dalam detail kodingan fungsi tersebut. Kemudian **Step Into** akan membawa penelusuran masuk ke dalam lingkup internal dari fungsi yang sedang dipanggil pada baris tersebut. Jika baris kode memanggil fungsi dari *library* pihak ketiga (seperti **Timber** atau **NavController**), Debugger akan membuka file sumber fungsi tersebut dan menunjukkan proses eksekusi di dalamnya langkah-demi-langkah. Dan terakhir Step Out, Fitur ini bertindak sebagai pintu keluar dari proses penelusuran yang sedang berlangsung. Jika ingin segera kembali ke alur kode utama, **Step Out** akan memerintahkan Debugger untuk menyelesaikan sisa eksekusi fungsi saat ini secepat mungkin dan langsung mengembalikan fokus penelusuran ke baris kode di mana fungsi tersebut pertama kali dipanggil.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya.

A. Pembahasan

Application Class dalam arsitektur aplikasi Android merupakan kelas fundamental yang bertindak sebagai induk dari seluruh komponen aplikasi dan menjadi titik awal eksekusi sistem sebelum *Activity*, *Service*, atau *Broadcast Receiver* dijalankan. Secara teknis, kelas ini adalah turunan dari kelas `android.app.Application` yang diciptakan oleh sistem operasi Android segera setelah proses aplikasi dimulai. Fungsi utamanya adalah sebagai wadah sentral untuk melakukan inisialisasi global, di mana pengembang dapat menyiapkan berbagai pustaka pihak ketiga seperti library **Timber** untuk logging, Database Room, hingga konfigurasi *Dependency Injection* secara satu pintu agar tersedia di seluruh bagian proyek. Selain itu, Application Class berperan sebagai penyedia *Global Context* yang memiliki siklus hidup (*lifecycle*). Class ini juga memungkinkan pengembang untuk memantau status aplikasi secara menyeluruh, seperti mendeteksi kapan aplikasi berpindah dari latar belakang ke latar depan guna mengatur penggunaan sumber daya secara lebih efisien. Agar dapat berfungsi sebagaimana mestinya, kelas ini harus didaftarkan secara eksplisit di dalam file **AndroidManifest.xml** melalui atribut `android:name`

TAUTAN GIT

Berikut adalah tautan untuk *source code* yang telah dibuat.

[Link Github](#)